

CSE 203: Data Structures

Elementary Data Structures

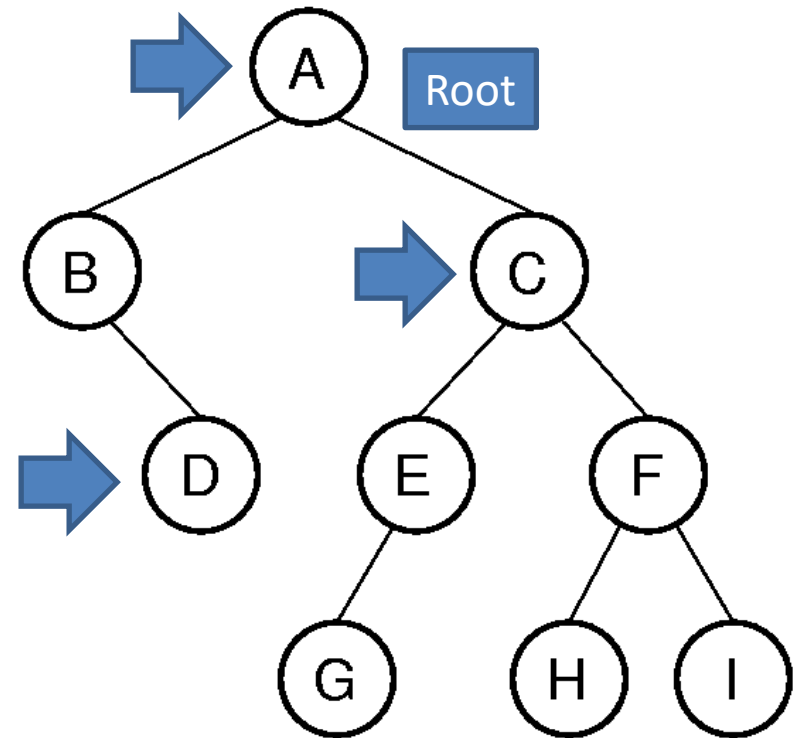
Binary Trees

Binary Trees

A binary tree is made up of a finite set of nodes. This set either is empty or consists of a node called the root together with two binary trees, called the left and right subtrees, which are disjoint from each other and from the root.

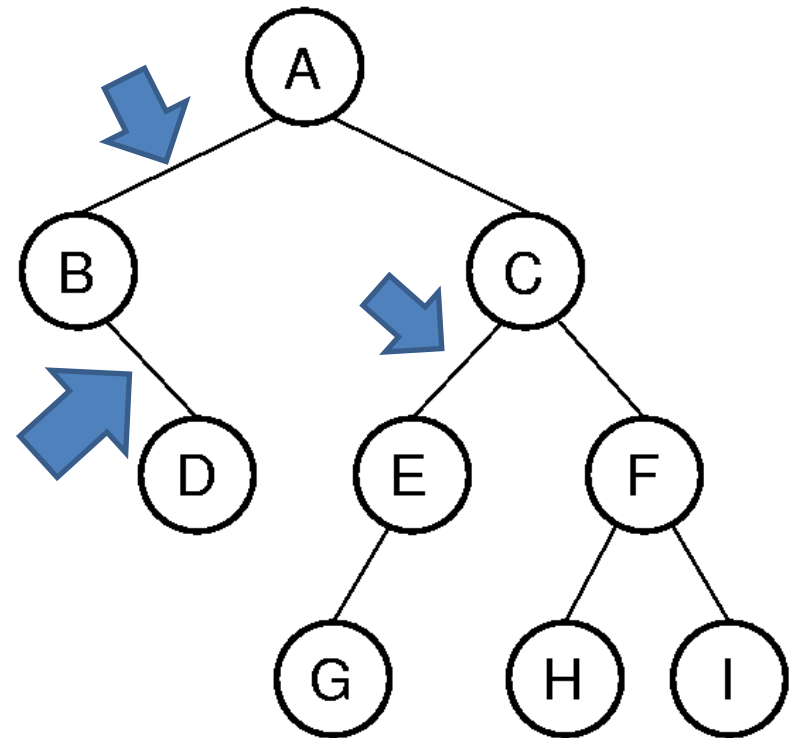
Binary Tree Example

Notation: Node,
children, edge,
parent, ancestor,
descendant, path,
depth, height, level,
leaf node, internal
node, subtree.



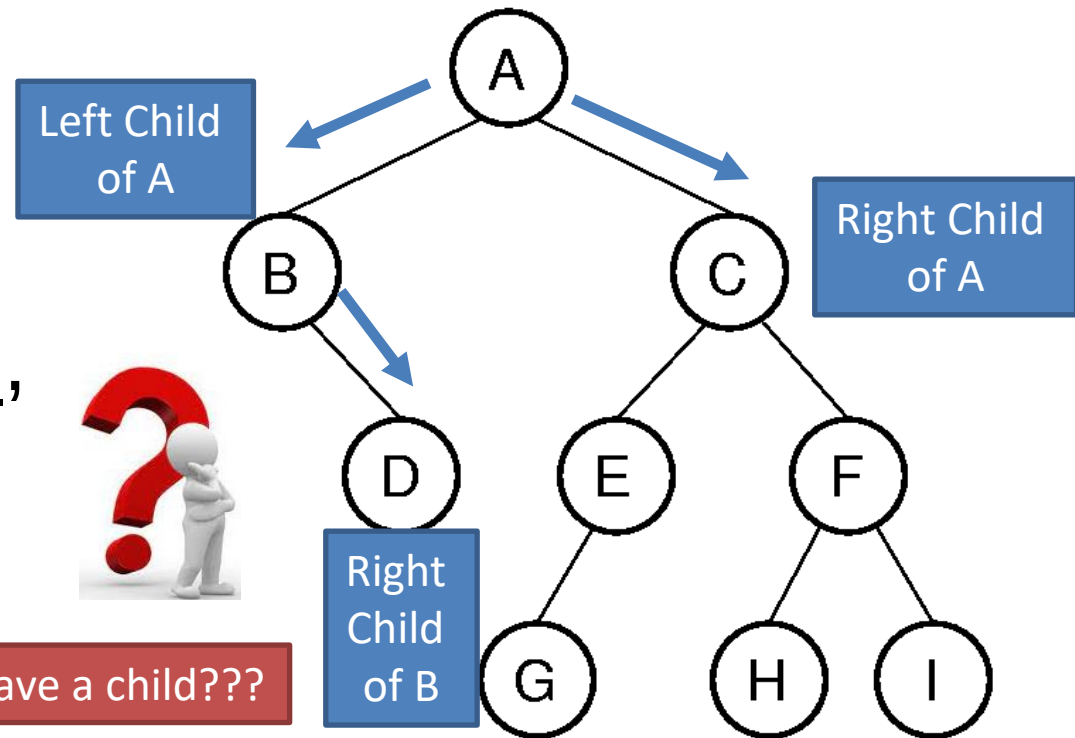
Binary Tree Example

Notation: Node, children, edge, parent, ancestor, descendant, path, depth, height, level, leaf node, internal node, subtree.



Binary Tree Example

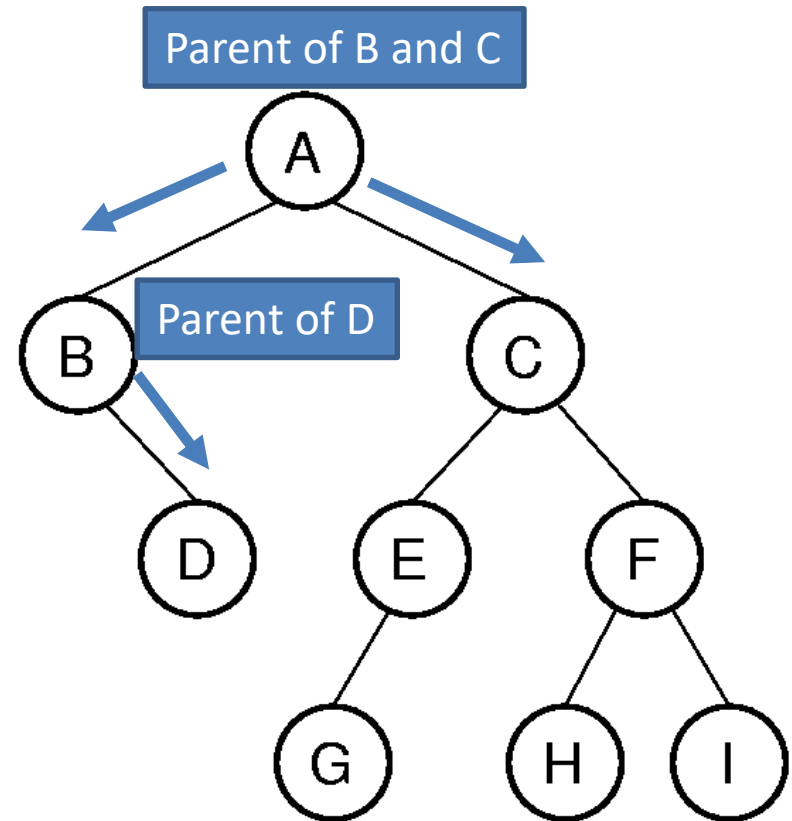
Notation: Node,
children, edge,
parent, ancestor,
descendant, path,
depth, height, level,
leaf node, internal
node, subtree.



B has an empty child!!

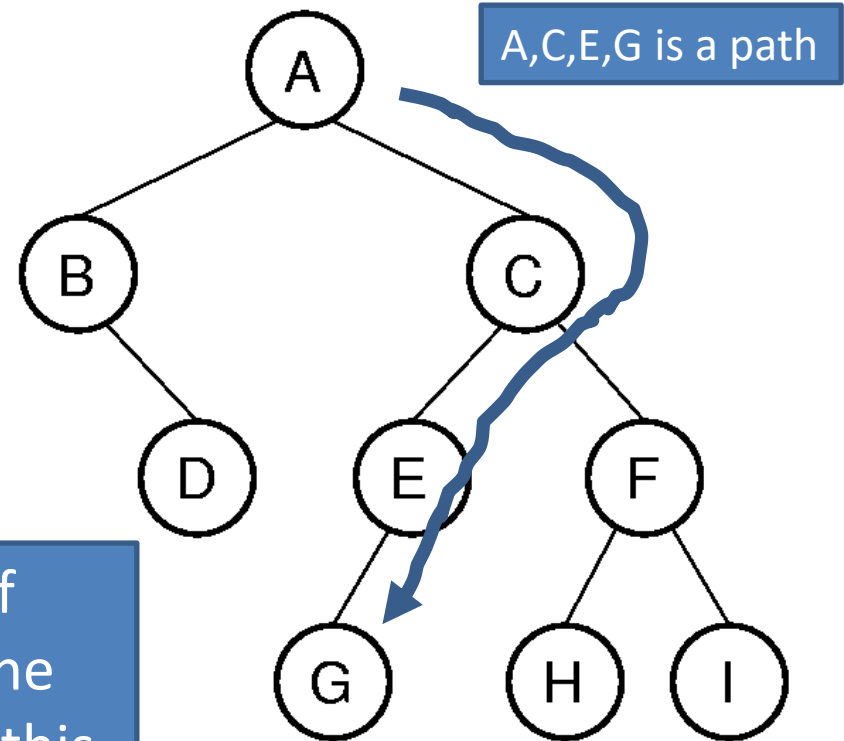
Binary Tree Example

Notation: Node,
children, edge,
parent, ancestor,
descendant, path,
depth, height, level,
leaf node, internal
node, subtree.



Binary Tree Example

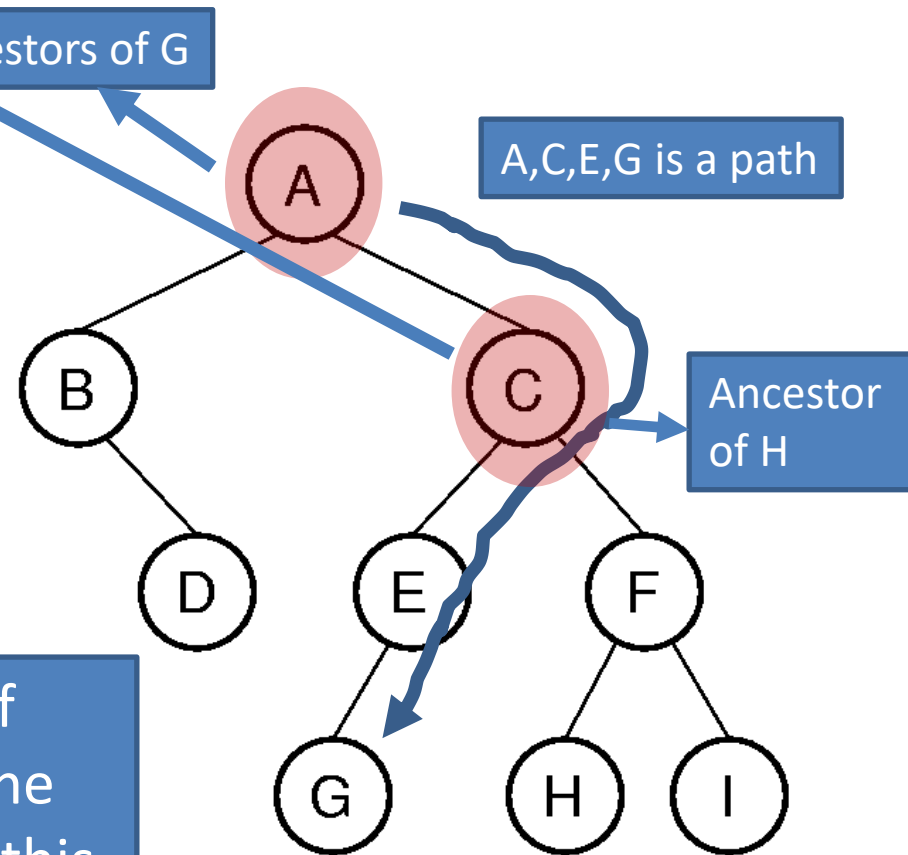
Notation: Node,
children, edge,
parent, ancestor,
descendant, path,
depth, height, level,
leaf node, internal
node, subtree.



If $n_1, n_2, \dots, n_k$ is a sequence of nodes in the tree such that n_i is the parent of n_{i+1} for $1 \leq i < k$, then this sequence is called a **path** from n_1 to n_k .

Binary Tree Example

Notation: Node,
children, edge,
parent, ancestor,
descendant, path,
depth, height, level,
leaf node, internal
node, subtree.

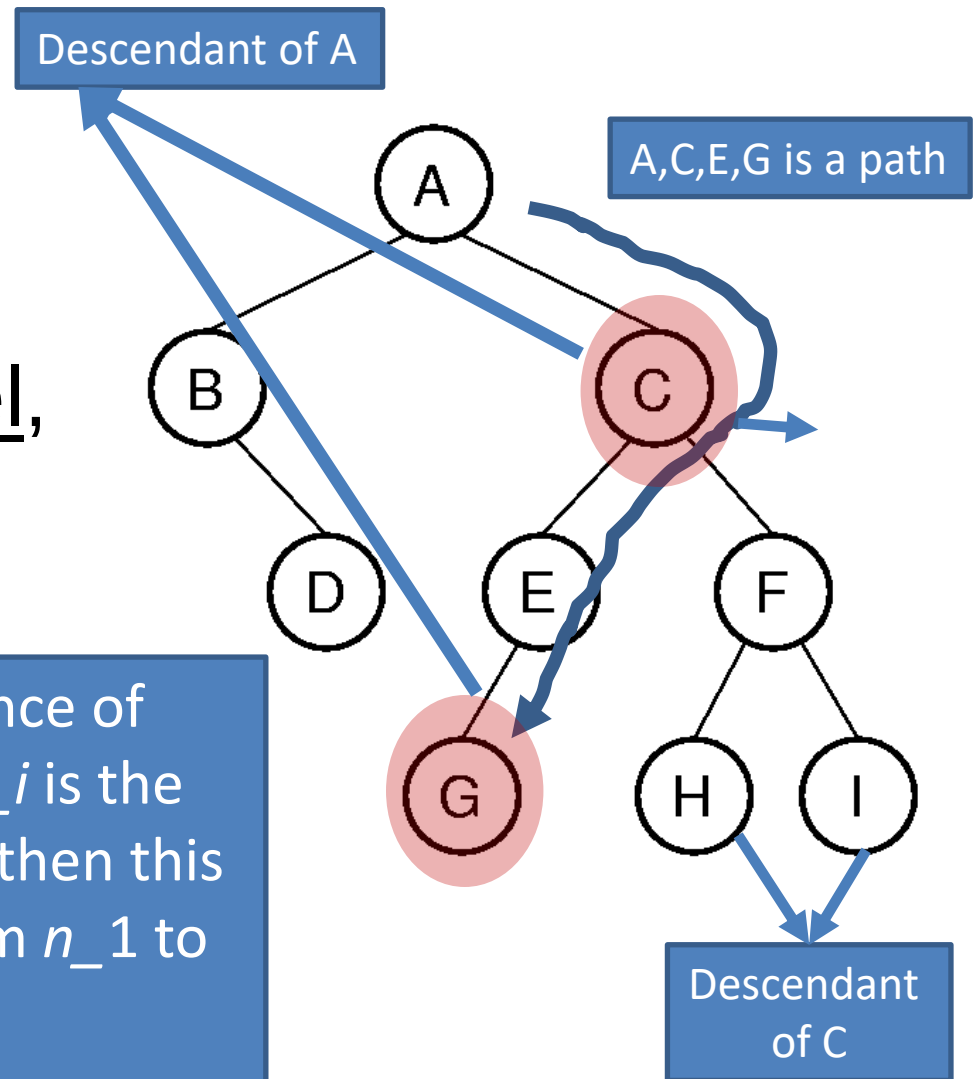


If $n_1, n_2, \dots, n_k$ is a sequence of nodes in the tree such that n_i is the parent of n_{i+1} for $1 \leq i < k$, then this sequence is called a **path** from n_1 to n_k .

Binary Tree Example

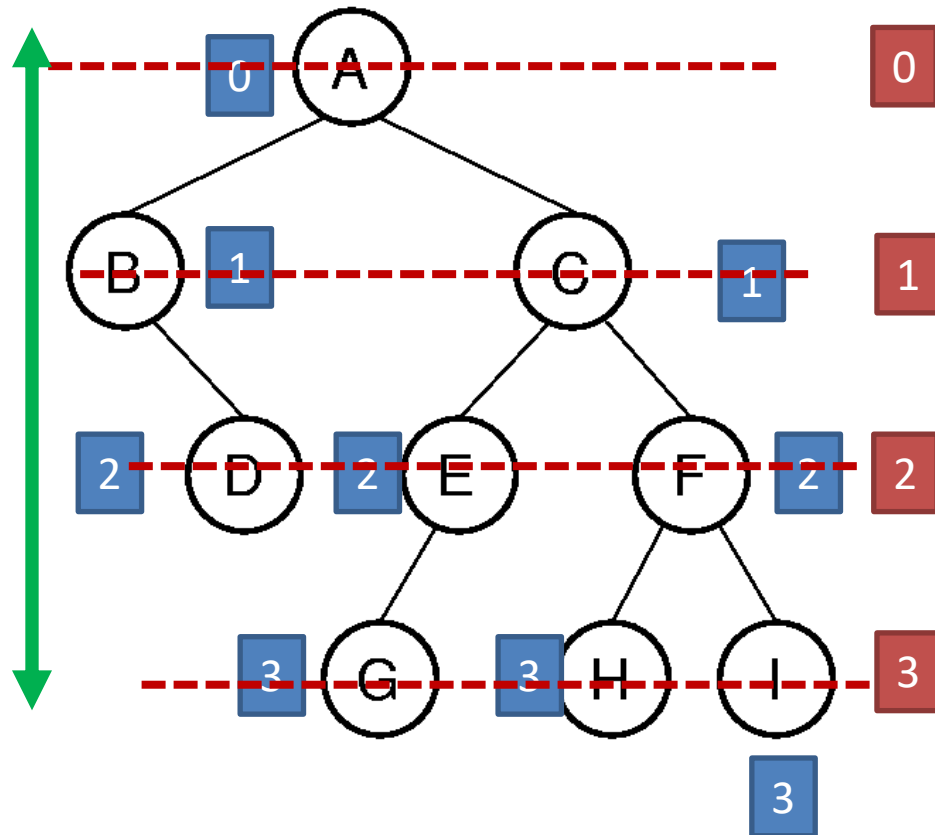
Notation: Node,
children, edge,
parent, ancestor,
descendant, path,
depth, height, level,
leaf node, internal
node, subtree.

If $n_1, n_2, \dots, n_k$ is a sequence of nodes in the tree such that n_i is the parent of n_{i+1} for $1 \leq i < k$, then this sequence is called a **path** from n_1 to n_k .



Binary Tree Example

Notation: Node,
children, edge,
parent, ancestor,
descendant, path,
depth, height, level,
leaf node, internal
node, subtree.



Depth of a node...

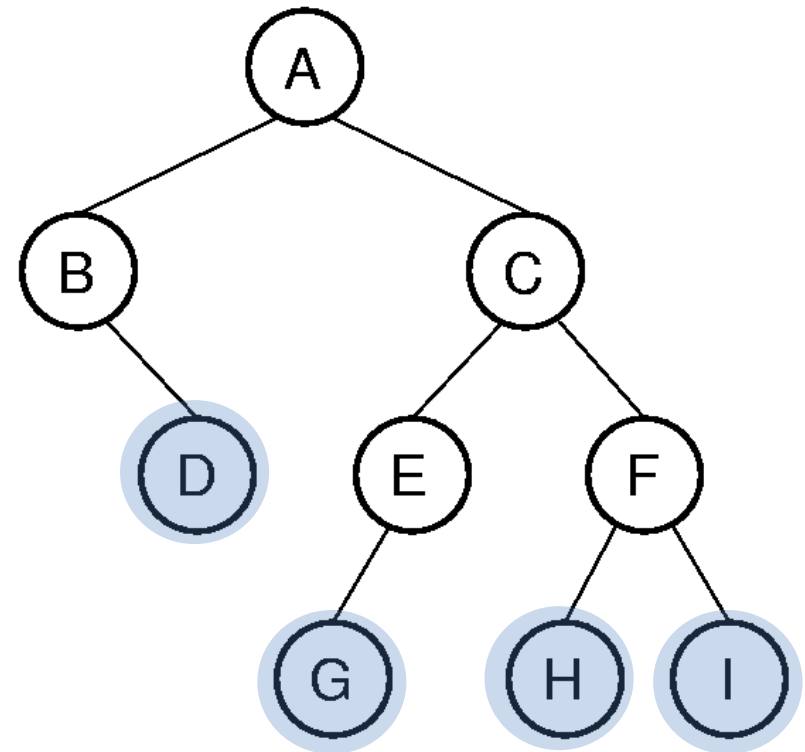
Level of a node...

Height of a tree

Binary Tree Example

Notation: Node,
children, edge, parent,
ancestor, descendant,
path, depth, height,
level, leaf node,
internal node, subtree.

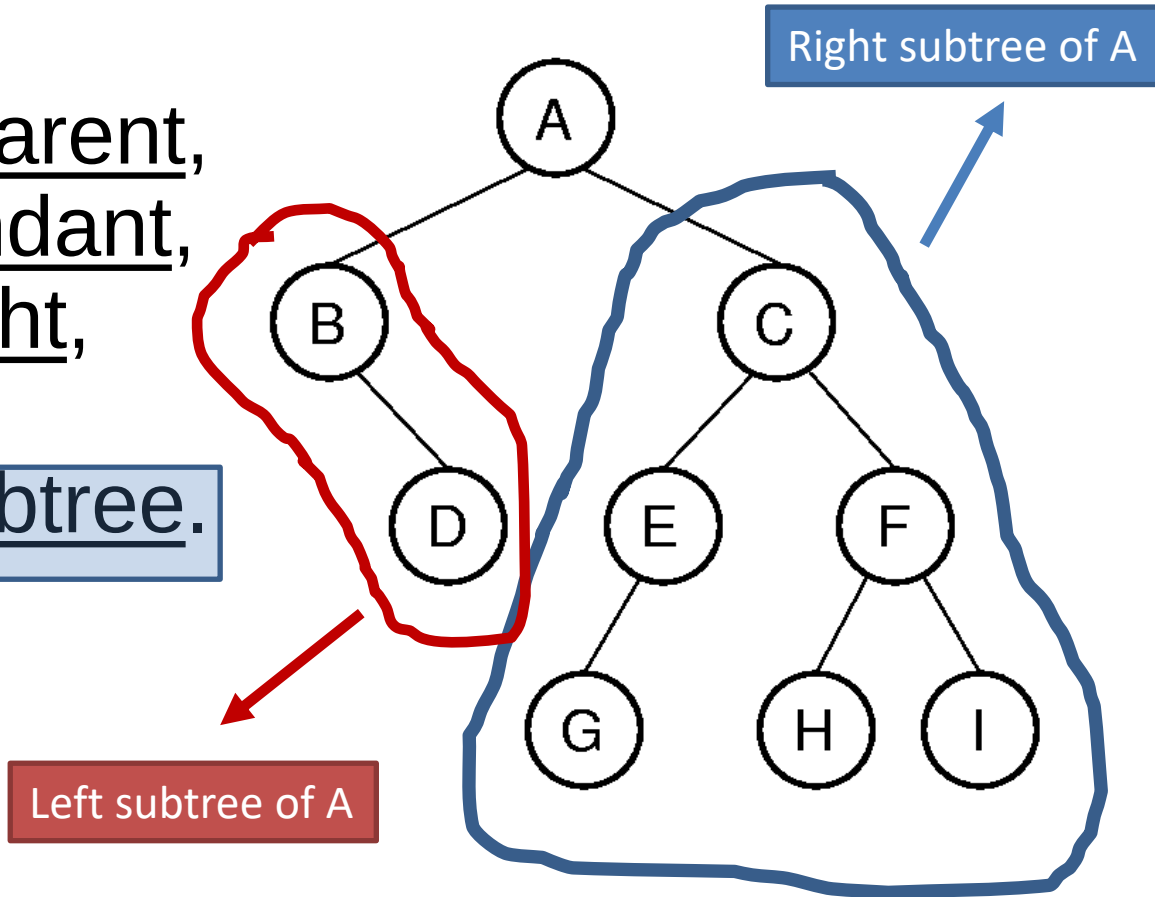
Internal node: At least one non-empty children



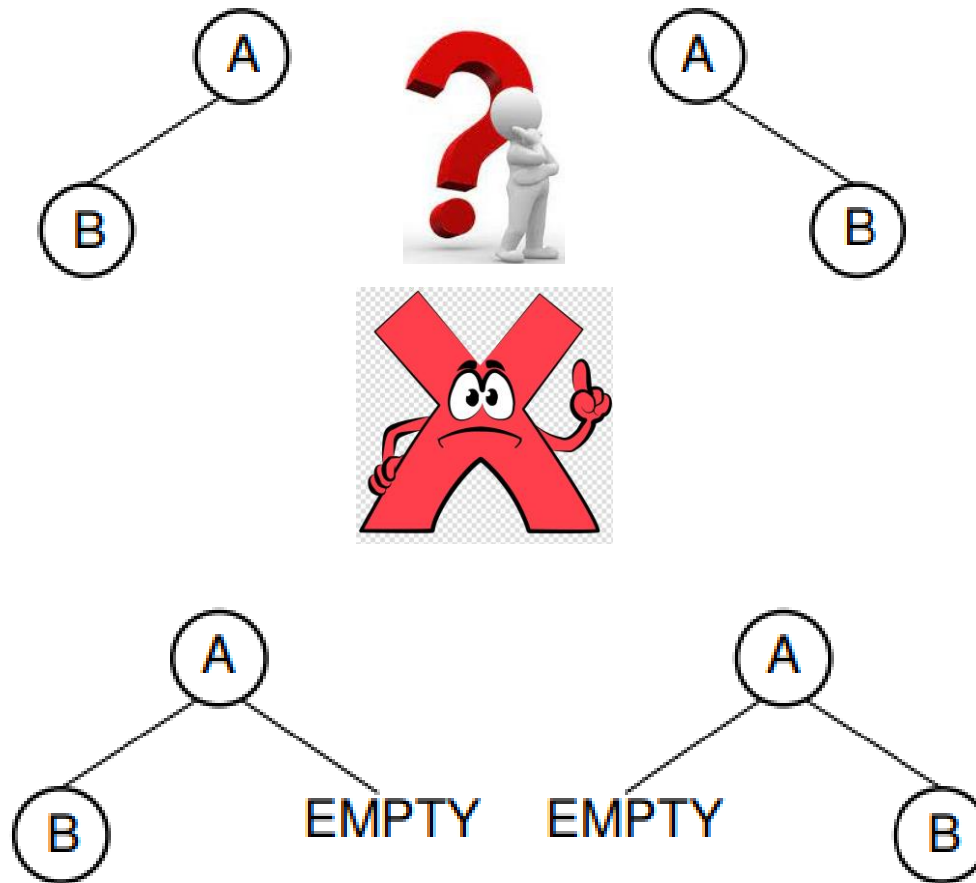
Leaf node: nodes having both children empty

Binary Tree Example

Notation: Node,
children, edge, parent,
ancestor, descendant,
path, depth, height,
level, leaf node,
internal node, subtree.



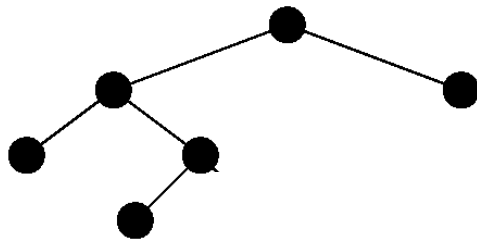
Are the following two trees same?



Full and Complete Binary Trees

Full binary tree: Each node is either a leaf or internal node with exactly two non-empty children.

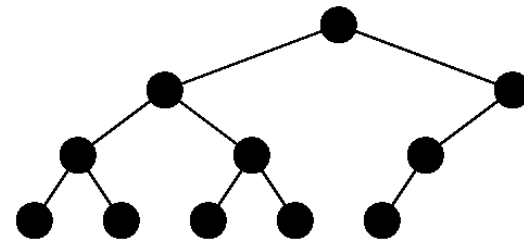
Complete binary tree: If the height of the tree is d , then all leaves except possibly level $d-1$ are completely full. The bottom level has all nodes to the left side.



(a)

Full

Not anymore



(b)

Complete

Not anymore

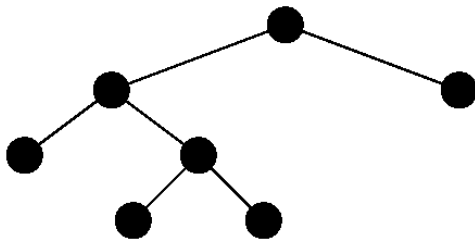
Full and Complete Binary Trees

Full binary tree: Each node is either a leaf or internal node with exactly two non-empty children.

Complete binary tree: If the height of the tree is d , then all leaves except possibly level $d-1$ are completely full. The bottom level has all nodes to the left side.

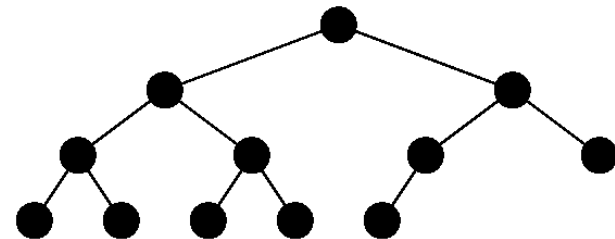
One Crucial Observation

NOT
COMPLETE



(a)

Full



(b)

Complete

NOT FULL

Full Binary Tree Theorem (1)

Theorem: The number of leaves in a non-empty full binary tree is one more than the number of internal nodes.

Proof (by Mathematical Induction):

Base case: A full binary tree with 1 internal node must have two leaf nodes.

Induction Hypothesis: Assume any full binary tree T containing $n-1$ internal nodes has n leaves.

Full Binary Tree Theorem (2)

Induction Step:

We want to prove the following here:

If $n-1$ internal leaves means n leaves

Then n internal leaves would mean $n+1$ leaves

[Recall that this will establish the chain:

1 IN means 2 L

So, 2 IN means 3 L

So, 3 IN means 4

And so on...]

Full Binary Tree Theorem (2)

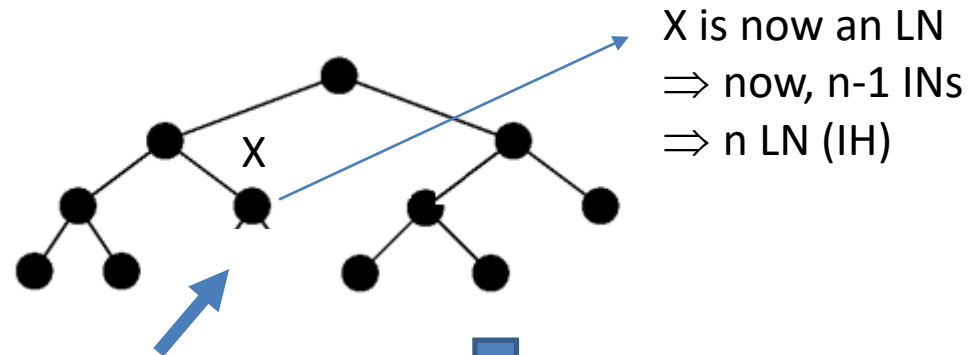
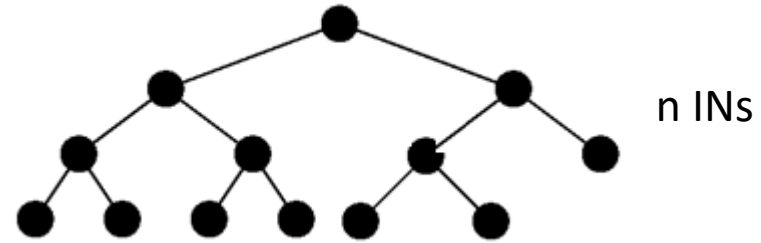
Induction Step: Given tree **T** with n internal nodes, pick internal node I with two leaf children. Remove I 's children, call resulting tree **T'**.

Clearly, **T'** is a full binary tree with $n-1$ internal nodes

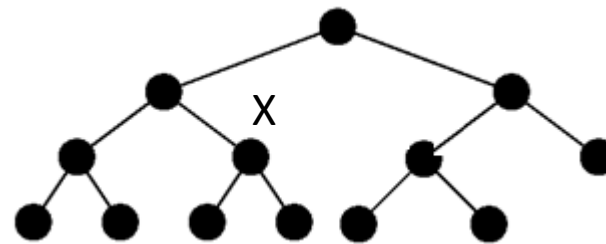
By induction hypothesis, it has n leaves.

Now:

Restore I 's two children. The number of internal nodes has now gone up by 1 to reach n . The number of leaves has also gone up by 1.



make X
a LN



X is now an IN
 $\Rightarrow$ now, n INs
How many LNs?
 X : IN $\Rightarrow$ LN
2 LNs added
 $\Rightarrow$ LN increases by 1
 $\Rightarrow n+1$ (from IH)

IN = Internal Node

LN = Leaf Node

IH = Induction Hypothesis

Why Full Binary Tree Theorem is useful?

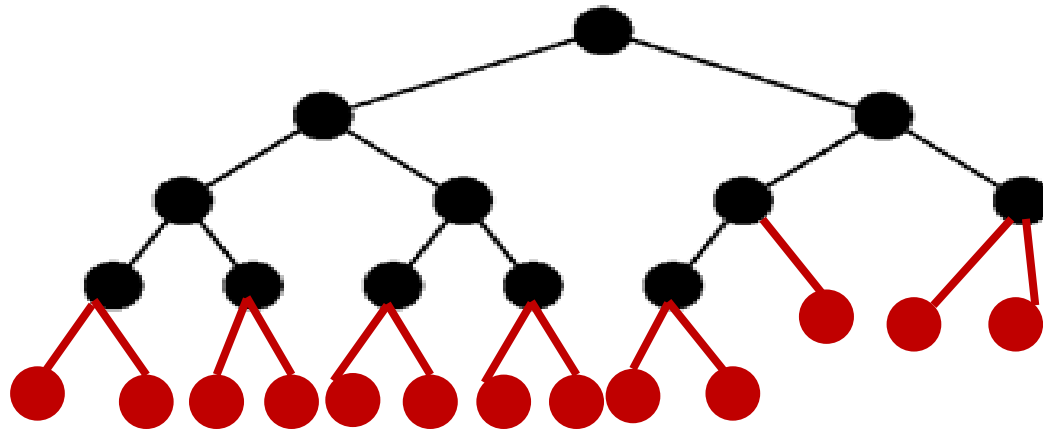
- Some binary tree implementations store data only at the leaf nodes, using the internal nodes to provide structure to the tree.
- More generally, binary tree implementations might require some amount of space for internal nodes, and a different amount for leaf nodes.

Full Binary Tree Corollary

Theorem: The number of empty subtrees in a non-empty binary tree is one more than the number of nodes in the tree.

Proof :

- Take an arbitrary binary tree T and replace every empty subtree with a leaf node. Call the new tree T_0 .
- All nodes originally in T will be internal nodes in T_0 (Why?)
- T_0 is a full binary tree, because every internal node of T now must have two children in T_0 , and each leaf node in T must have two children in T_0 (the leaves just added).
- The Full Binary Tree Theorem tells us that the number of leaves in a full binary tree is one more than the number of internal nodes.
- Thus, the number of new leaves that were added to create T_0 is one more than the number of nodes in T .
- Each leaf node in T_0 corresponds to an empty subtree in T . Thus, the number of empty subtrees in T is one more than the number of nodes in T .



Full Binary Tree Corollary

Theorem: The number of empty subtrees in a non-empty binary tree is one more than the number of nodes in the tree.

Another Proof :

We prove that a binary tree with n nodes has $n+1$ empty subtrees

By definition, every node in binary tree **T** has two children (some may be empty)

⇒ A total of $2n$ children in a tree of n nodes.

⇒ Every node except the root node has one parent.

⇒ A total of $n - 1$ nodes with parents.

⇒ In other words, there are $n - 1$ non-empty children.

⇒ Because the total number of children is $2n$, the remaining $n + 1$ children must be empty.

Binary Tree Node Class

156

Chap. 5 Binary Trees

```
// Binary tree node abstract class
template <typename E> class BinNode {
public:
    virtual ~BinNode() {} // Base destructor

    // Return the node's value
    virtual E& element() = 0;

    // Set the node's value
    virtual void setElement(const E&) = 0;

    // Return the node's left child
    virtual BinNode* left() const = 0;

    // Set the node's left child
    virtual void setLeft(BinNode*) = 0;

    // Return the node's right child
    virtual BinNode* right() const = 0;

    // Set the node's right child
    virtual void setRight(BinNode*) = 0;

    // Return true if the node is a leaf, false otherwise
    virtual bool isLeaf() = 0;
};
```

Figure 5.5 A binary tree node ADT.

Example 5.1 The preorder enumeration for the tree of Figure 5.1 is

ABDCEGFHL

The first node printed is the root. Then all nodes of the left subtree are printed (in preorder) before any node of the right subtree.

Alternatively, we might wish to visit each node *only after* we visit its children (and their subtrees). For example, this would be necessary if we wish to return all nodes in the tree to free store. We would like to delete the children of a node before deleting the node itself. But to do that requires that the children's children be deleted first, and so on. This is called a **postorder traversal**.

Example 5.2 The postorder enumeration for the tree of Figure 5.1 is

DBGEHIFCA.

An **inorder traversal** first visits the left child (including its entire subtree), then visits the node, and finally visits the right child (including its entire subtree). The

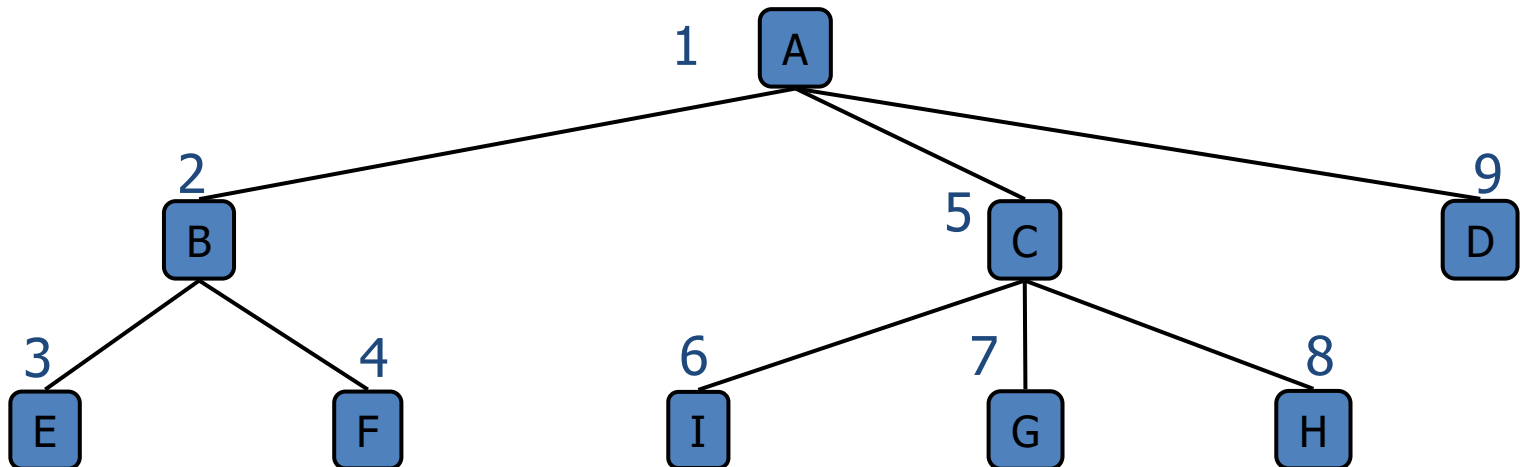
Traversal of Trees

- ◆ Any process for visiting the nodes in some order is called a traversal.
- ◆ A traversal of a tree T is a systematic way of visiting all the nodes of T
- ◆ Traversing a tree involves visiting the root and traversing its subtrees
- ◆ Mainly, there are the following traversal methods:
 - Preorder Traversal (applicable for any tree)
 - Postorder Traversal (applicable for any tree)
 - Inorder Traversal (of a binary tree)
- Any traversal that lists every node in the tree exactly once is called an enumeration of the tree's nodes.

Preorder Traversal

- In a preorder traversal, a node is visited before its descendants
- If a tree is ordered, then the subtrees are traversed according to the order of the children
- We assume a left to right order

Algorithm *preOrder(v)*
visit(v)
for each child *w* of *v*
preOrder(w)



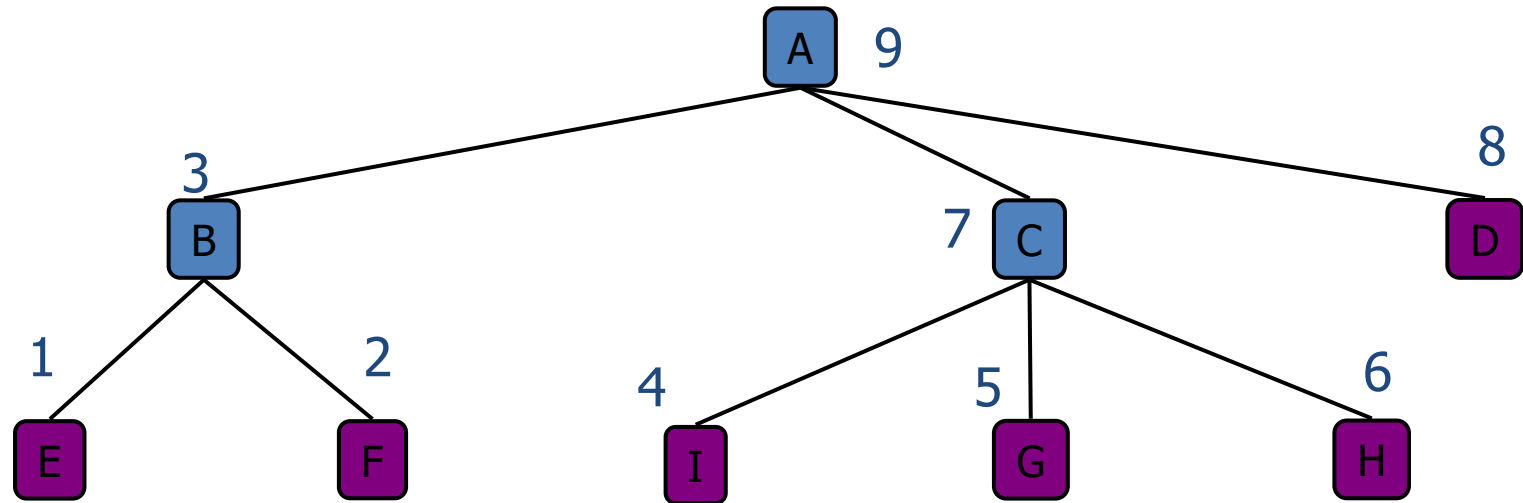
Preorder: ABEFCIGHD

Note: this is not a binary tree

Postorder Traversal

- In a postorder traversal, a node is visited after its descendants

Algorithm *postOrder(v)*
for each child *w* of *v*
 postOrder(w)
 visit(v)



Postorder: *EFBIGHCDA*

Note: this is not a binary tree

Inorder Traversal

- In an inorder traversal a node is visited after its left subtree and before its right subtree:
 - Traverse the **left subtree** in inorder.
 - Visit the **root**.
 - Traverse the **right subtree** in inorder.

Algorithm ***inOrder(v)***

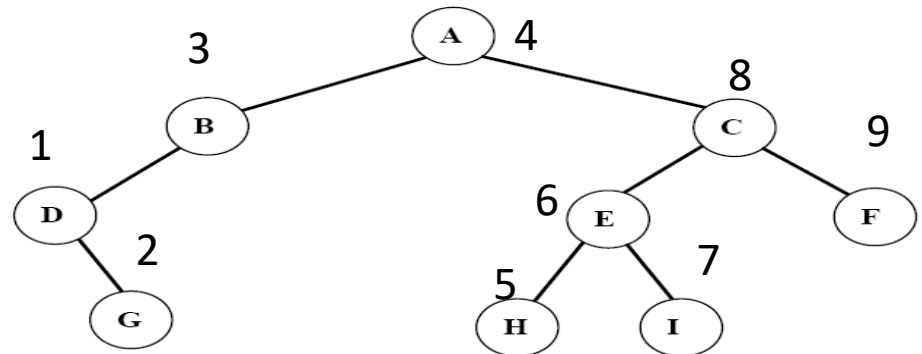
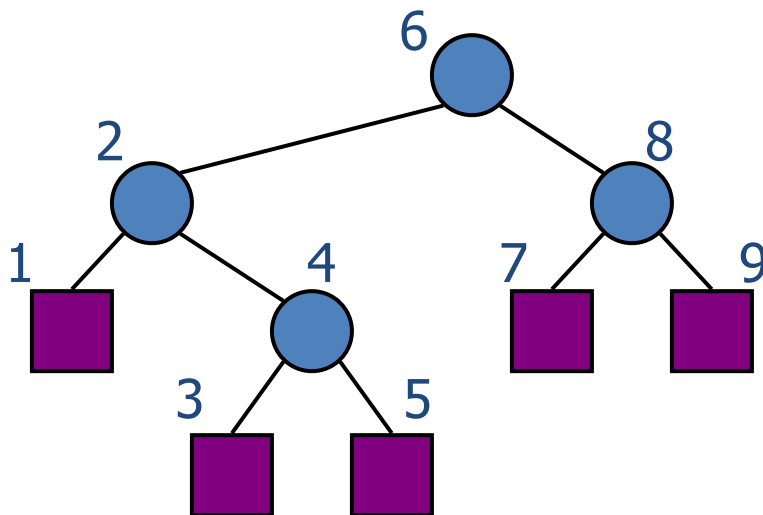
if *isInternal* (v)

inOrder (leftChild (v))

visit(v)

if *isInternal* (v)

inOrder (rightChild (v))

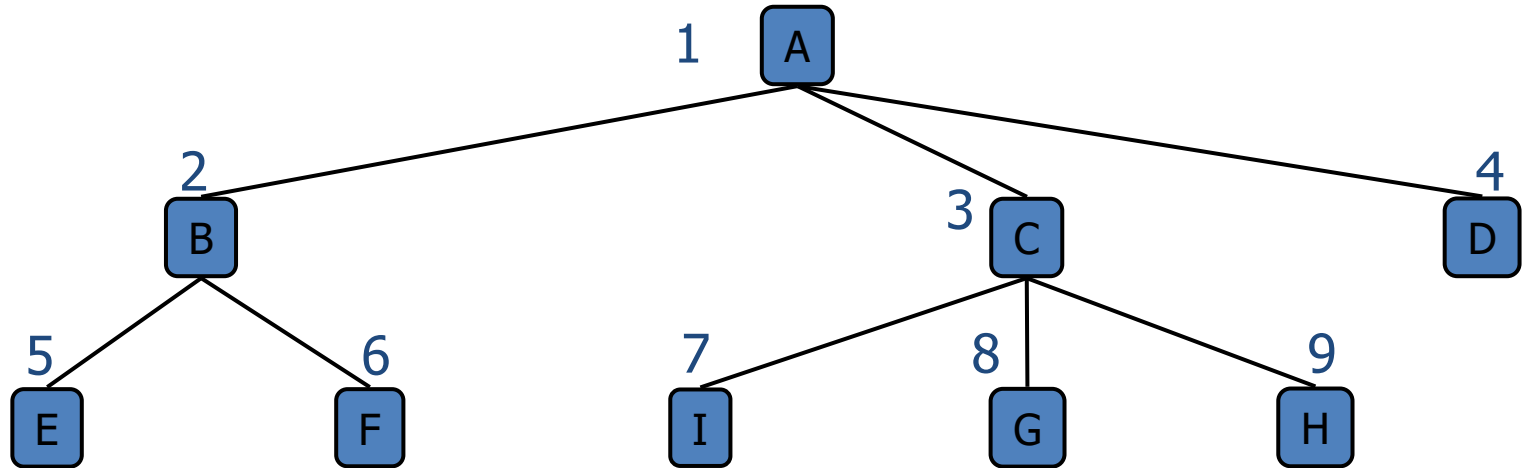


Inorder: *DGBAHEICF*

Note: Only applicable to a binary tree

Level Order Traversal

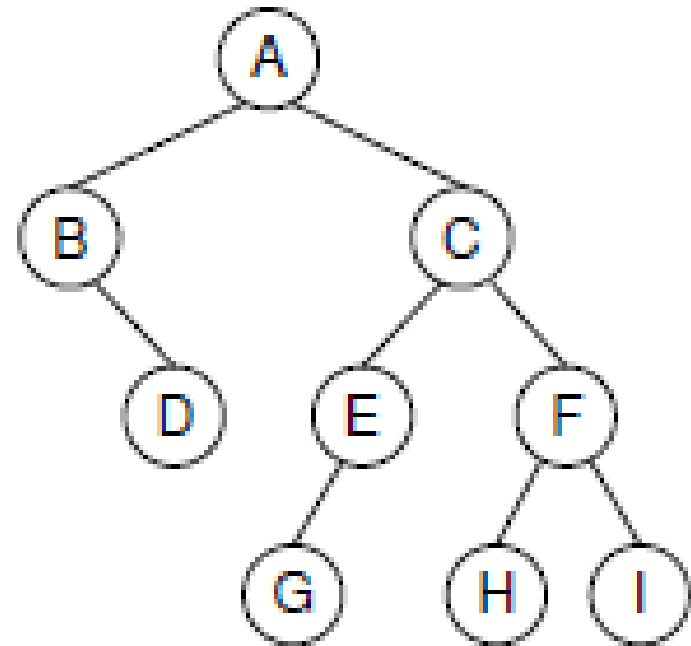
- In a level order traversal, every node on a level is visited before going to a lower level



Level order: *ABCDEFGH*

Pre-Order Traversal

1.
2.
3.
4.
5.
6.
7.
8.
9.



Pre-Order Traversal

1. A

2.

3.

4.

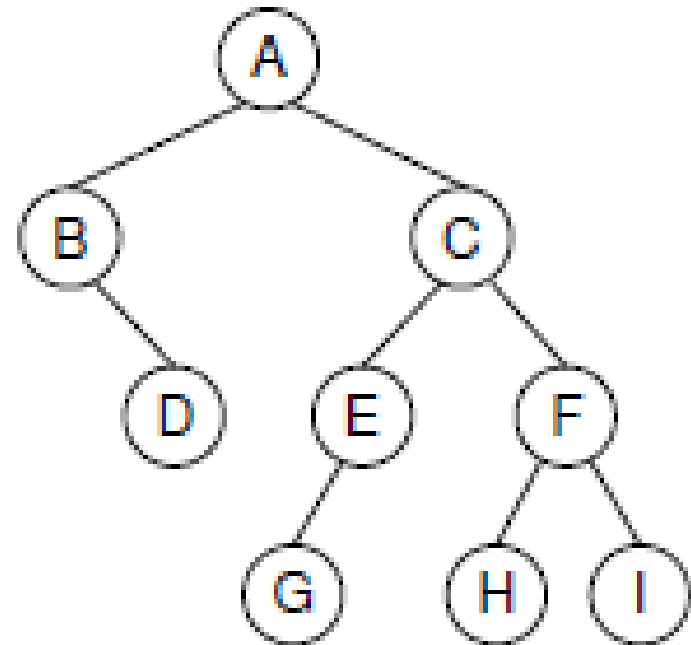
5.

6.

7.

8.

9.



Pre-Order Traversal

1. A

2. B

3. 

4.

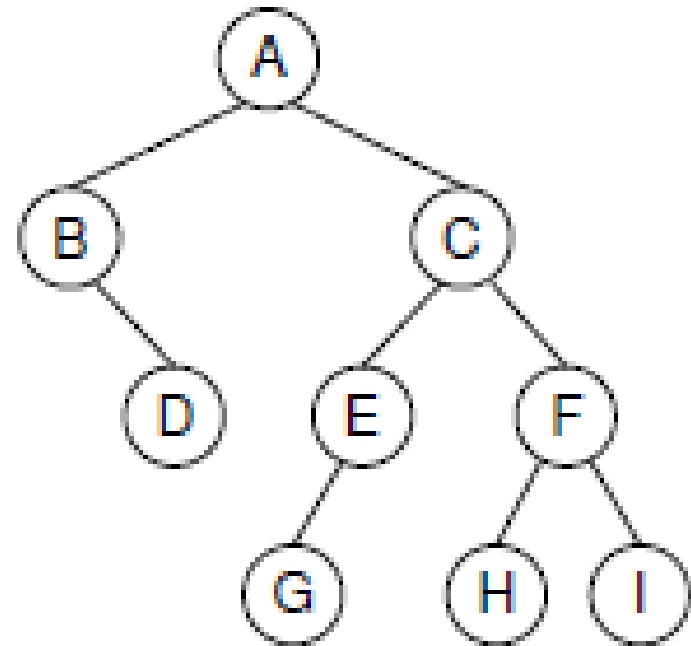
5.

6.

7.

8.

9.



Pre-Order Traversal

1. A

2. B

3. D

4. 

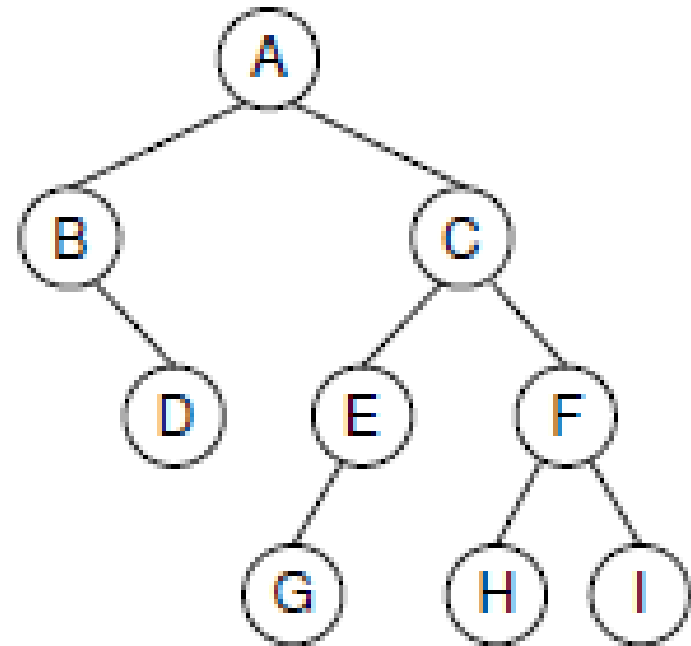
5.

6.

7.

8.

9.



Pre-Order Traversal

1. A

2. B

3. D

4. C

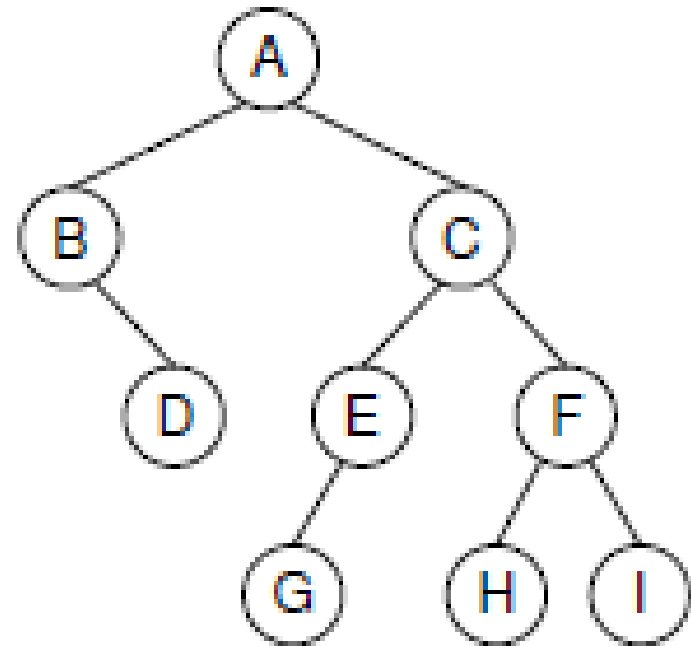
5. 

6.

7.

8.

9.



Pre-Order Traversal

1. A

2. B

3. D

4. C

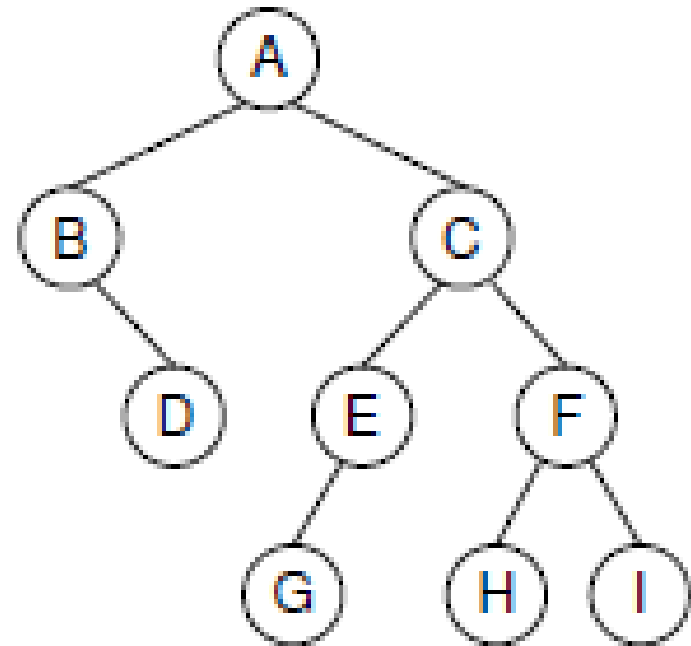
5. E

6. 

7.

8.

9.



Pre-Order Traversal

1. A

2. B

3. D

4. C

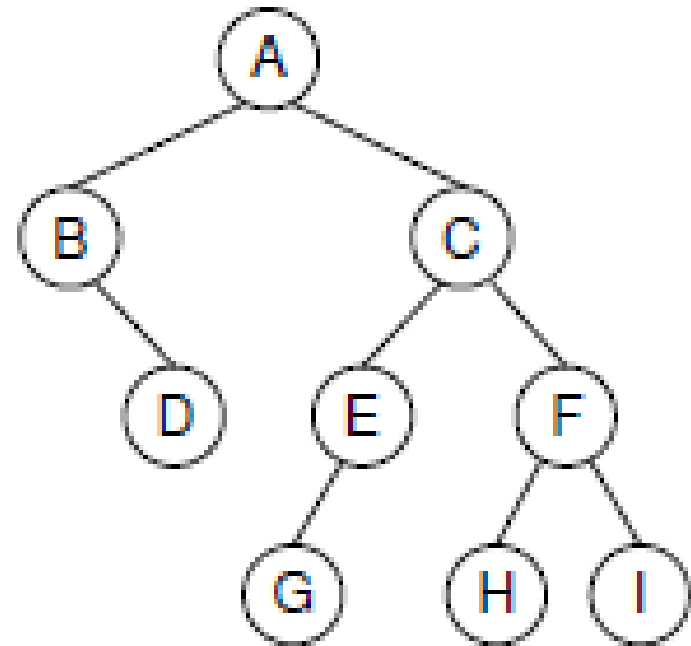
5. E

6. G

7. 

8. 

9. 



Pre-Order Traversal

1. A

2. B

3. D

4. C

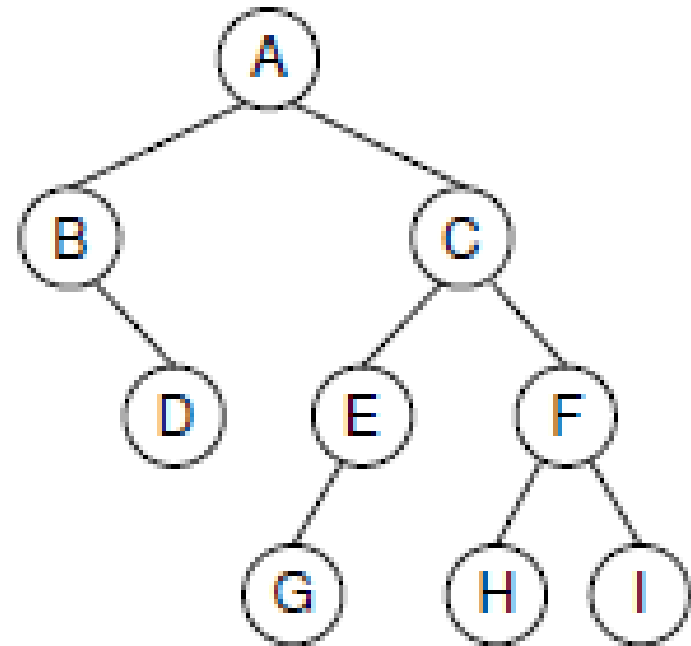
5. E

6. G

7. F

8. 

9. 



Pre-Order Traversal

1. A

2. B

3. D

4. C

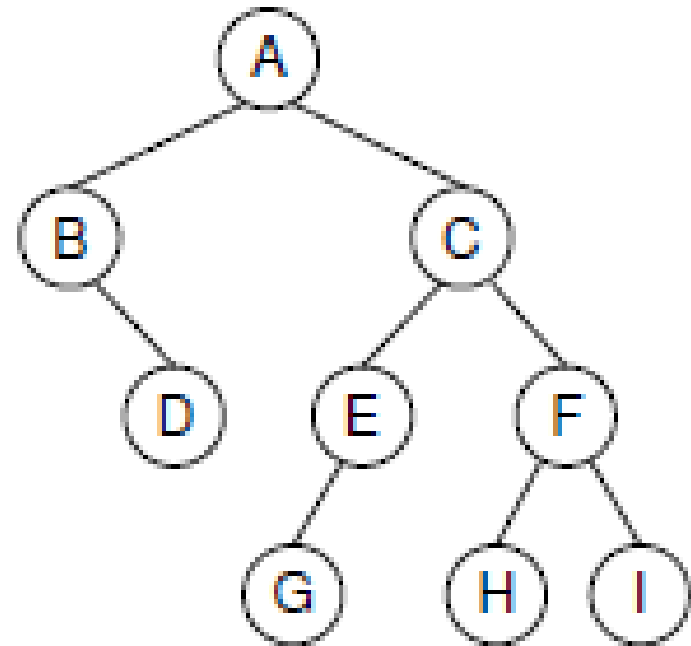
5. E

6. G

7. F

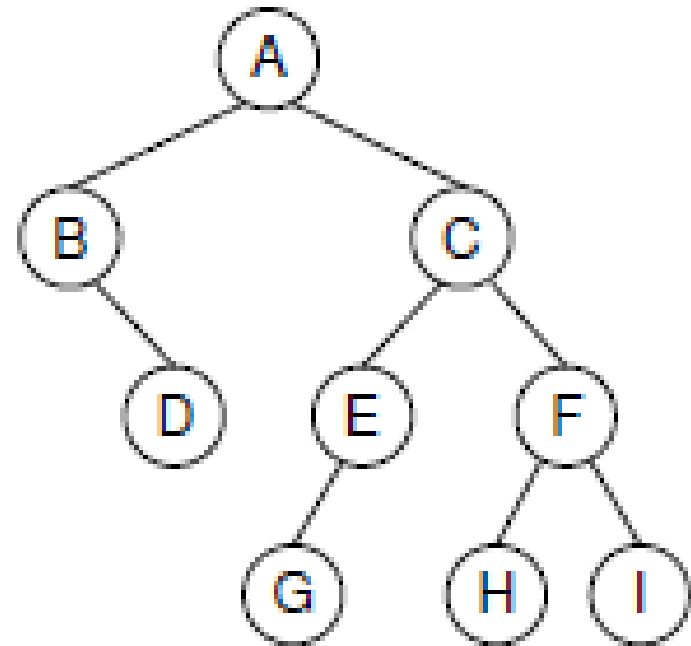
8. H

9. 



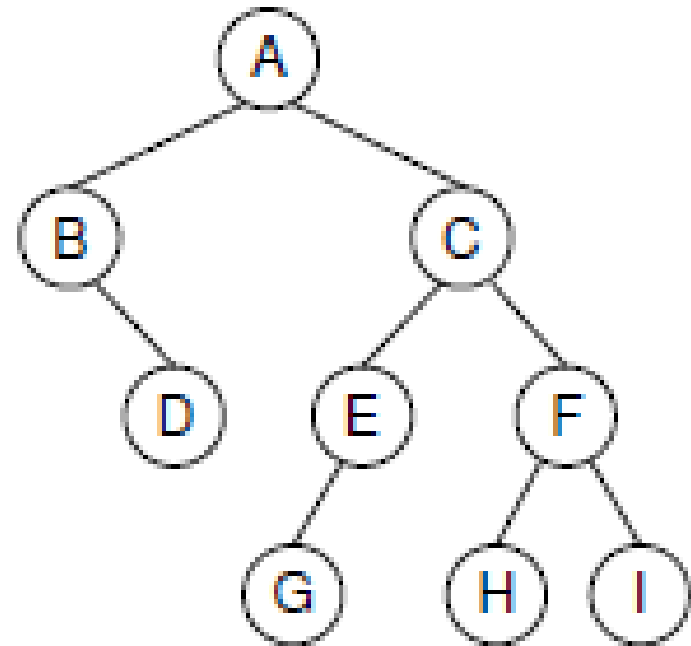
Pre-Order Traversal

1. A
2. B
3. D
4. C
5. E
6. G
7. F
8. H
9. I



Post-Order Traversal

1.
2.
3.
4.
5.
6.
7.
8.
9.



Post-Order Traversal

1. D

2.

3.

4.

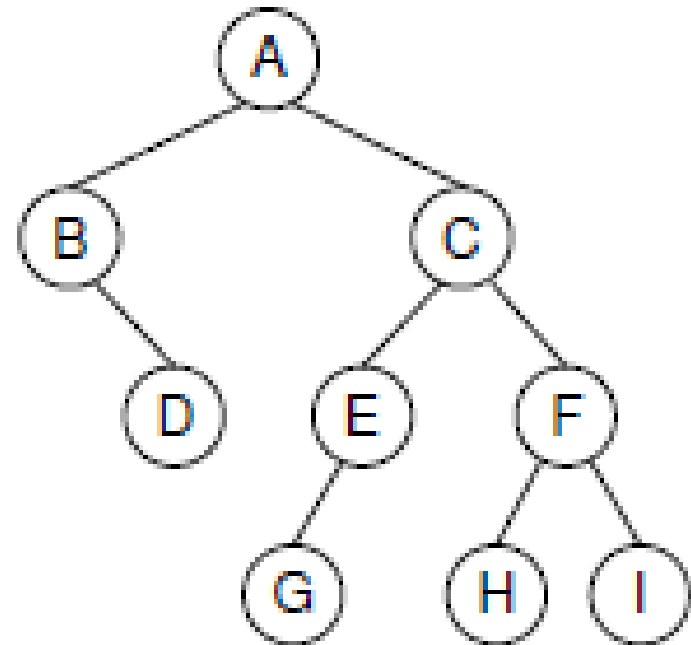
5.

6.

7.

8.

9.



Post-Order Traversal

1. D

2. B

3. 

4. 

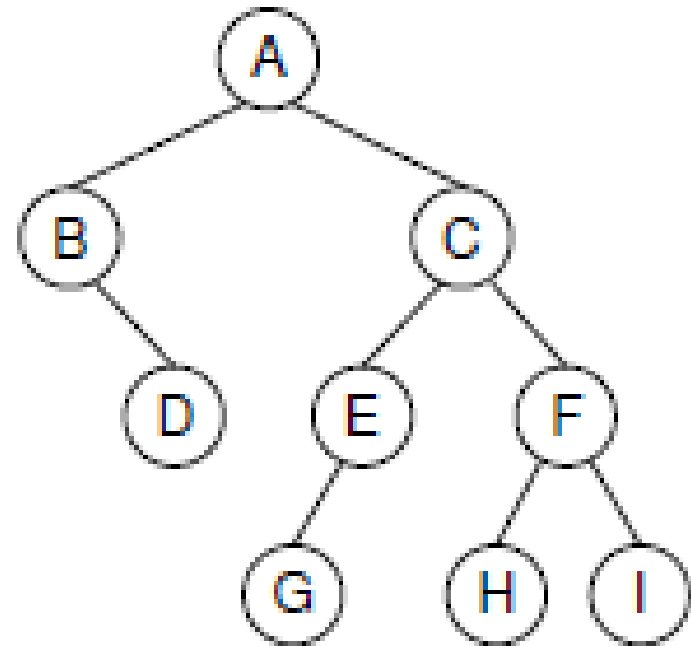
5. 

6. 

7. 

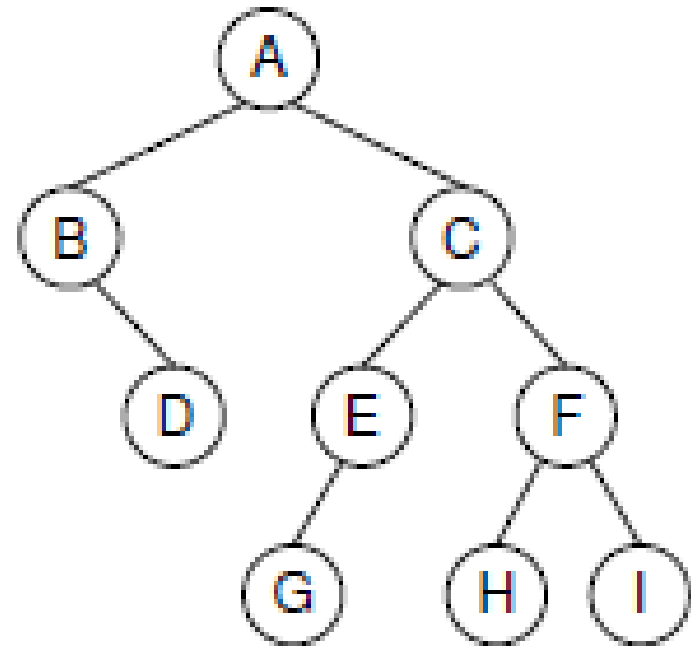
8. 

9. 



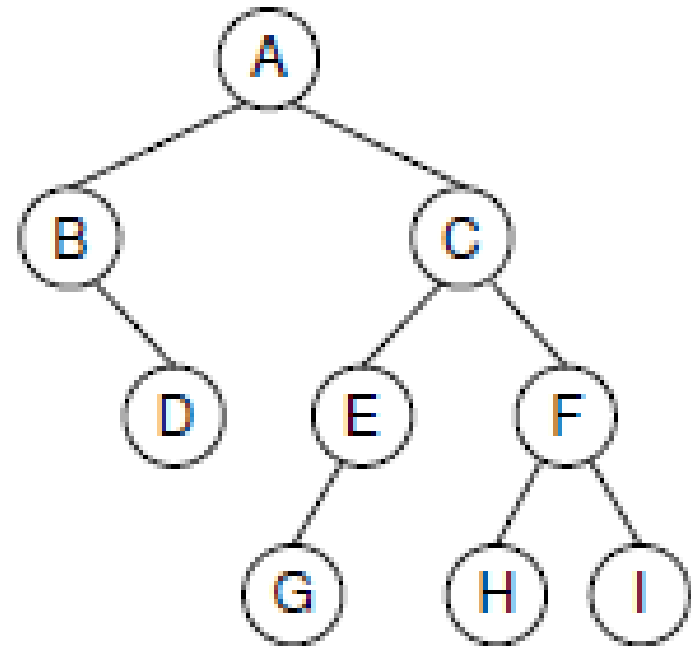
Post-Order Traversal

1. D
2. B
3. G
4. 
- 5.
- 6.
- 7.
- 8.
- 9.



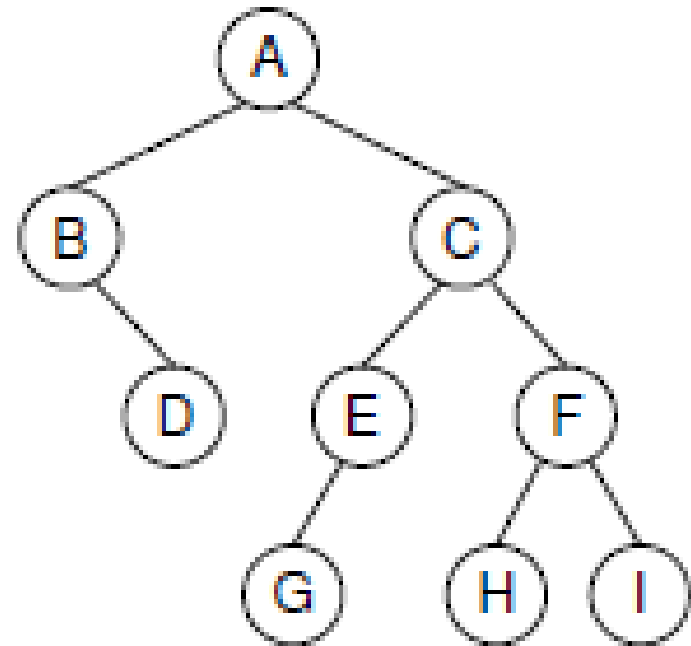
Post-Order Traversal

1. D
2. B
3. G
4. E
5.
6.
7.
8.
9.



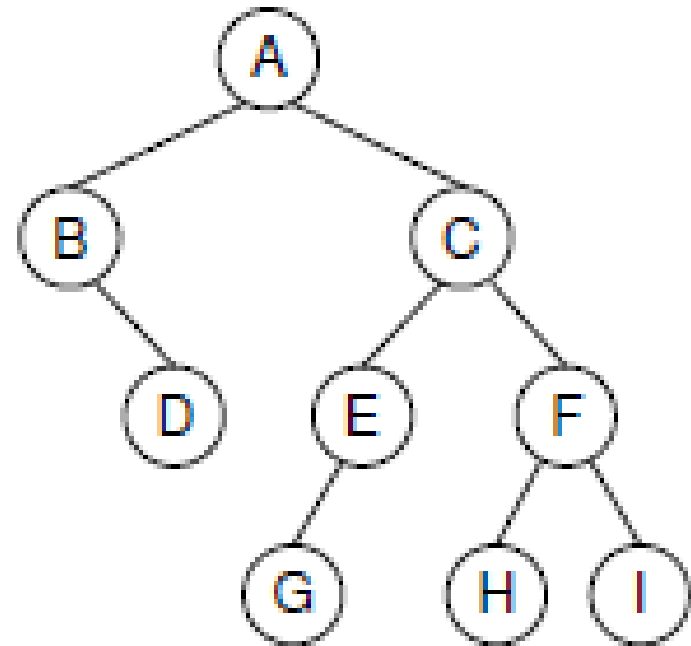
Post-Order Traversal

1. D
2. B
3. G
4. E
5. H
6. 
7. 
8. 
9. 



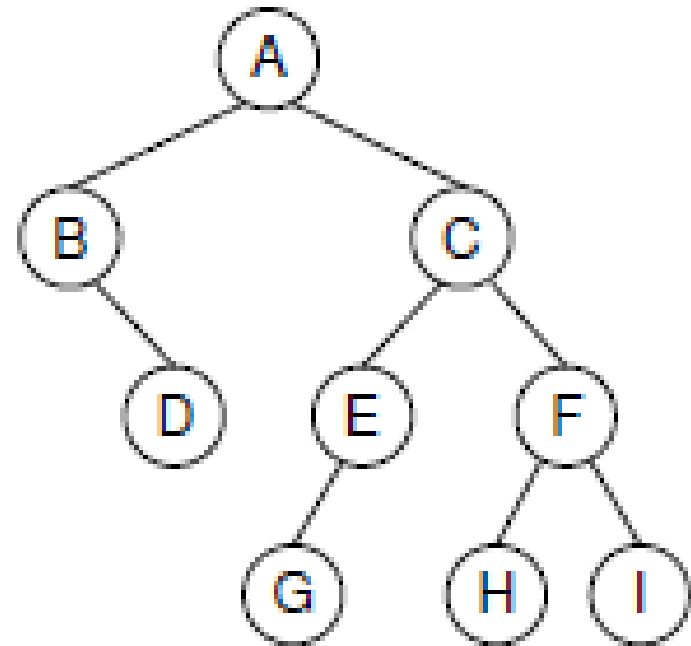
Post-Order Traversal

1. D
2. B
3. G
4. E
5. H
6. I
7. 
8. 
9. 



Post-Order Traversal

1. D
2. B
3. G
4. E
5. H
6. I
7. F
8. 
9. 



Post-Order Traversal

1. D

2. B

3. G

4. E

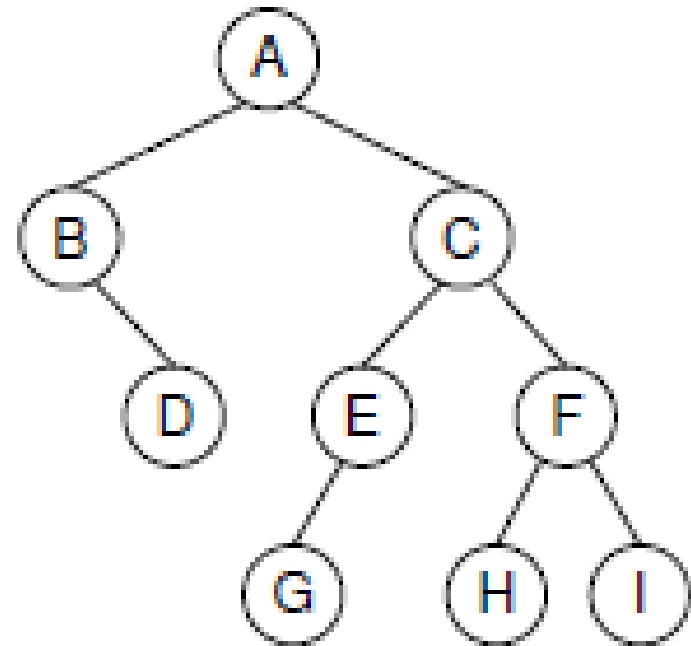
5. H

6. I

7. F

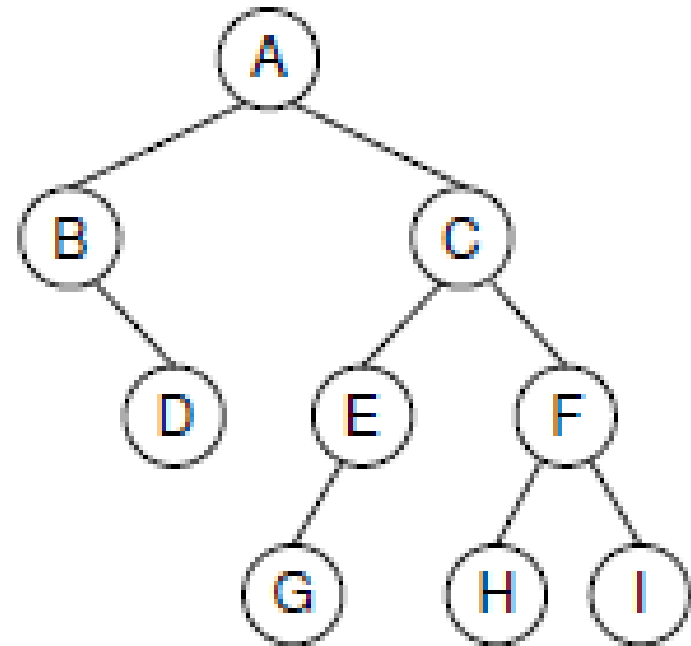
8. C

9. 



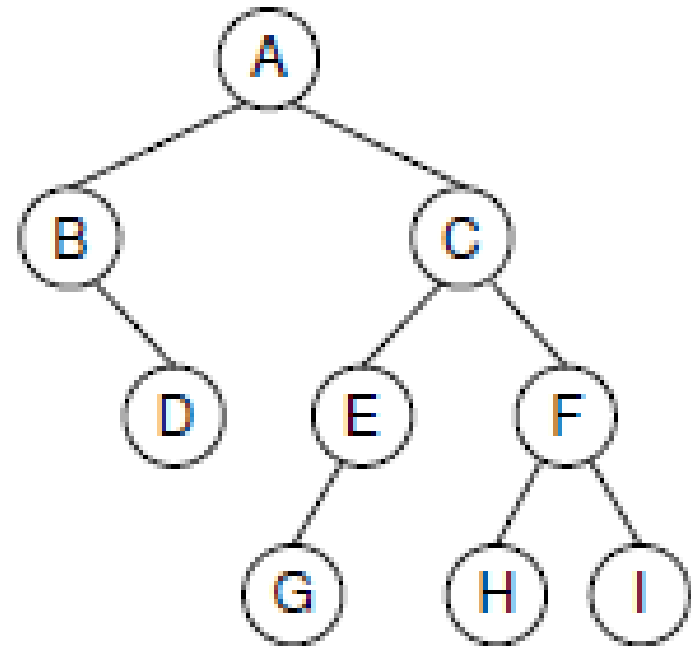
Post-Order Traversal

1. D
2. B
3. G
4. E
5. H
6. I
7. F
8. C
9. A



In-Order Traversal

- 1.
 - 2.
 - 3.
 - 4.
 - 5.
 - 6.
 - 7.
 - 8.
 - 9.
- 



In-Order Traversal

1. B

2.

3.

4.

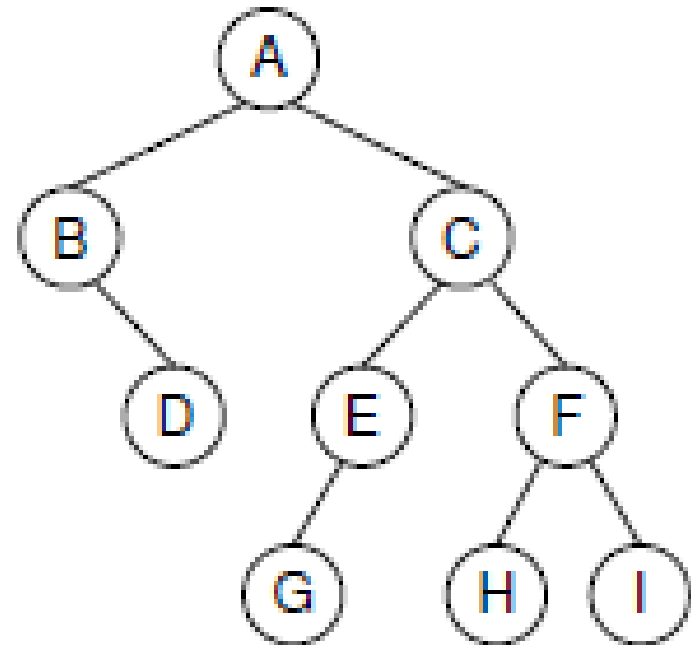
5.

6.

7.

8.

9.



In-Order Traversal

1. B

2. D

3. 

4.

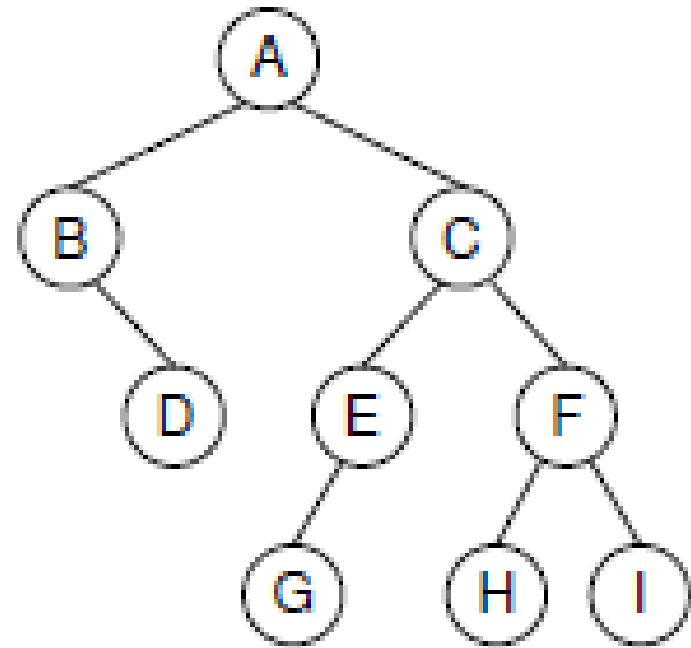
5.

6.

7.

8.

9.



In-Order Traversal

1. B

2. D

3. A

4.

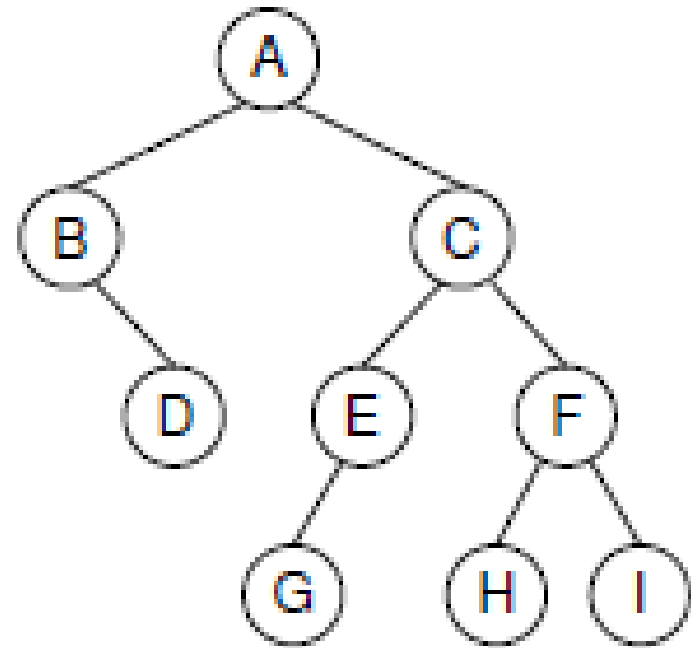
5.

6.

7.

8.

9.



In-Order Traversal

1. B

2. D

3. A

4. G

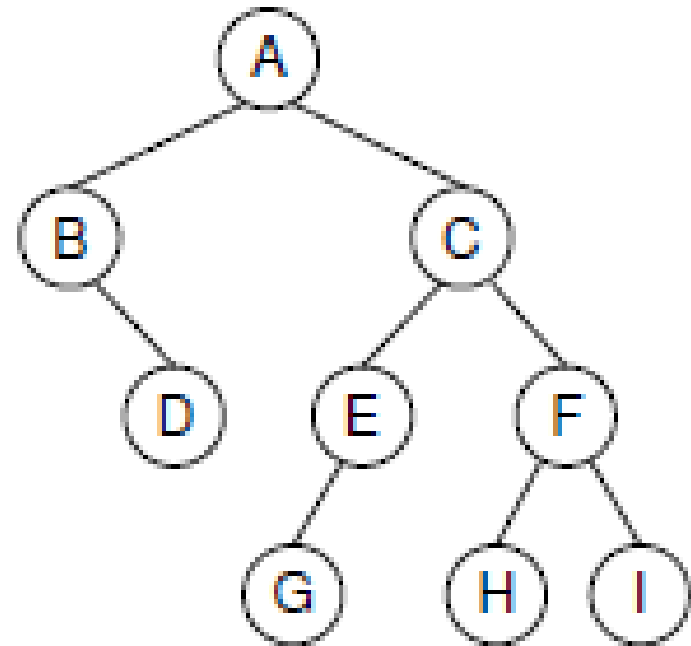
5. 

6.

7.

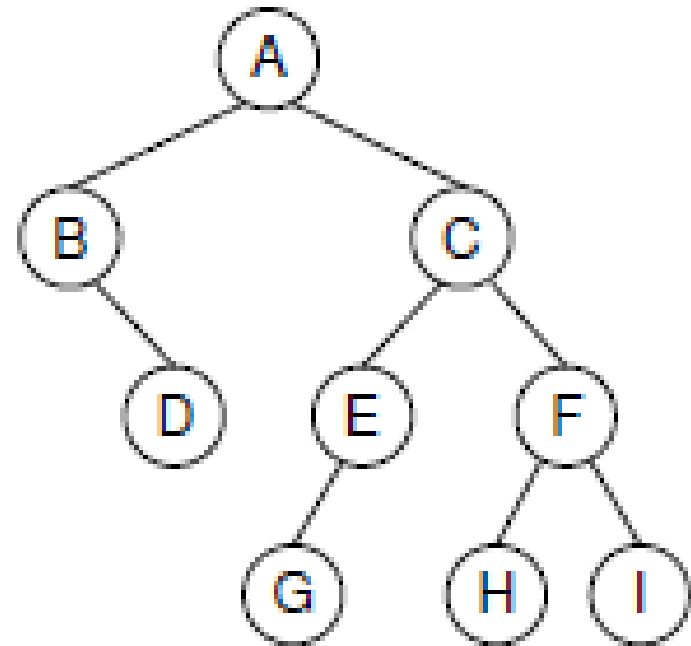
8.

9.



In-Order Traversal

1. B
2. D
3. A
4. G
5. E
6.
7.
8.
9.



In-Order Traversal

1. B

2. D

3. A

4. G

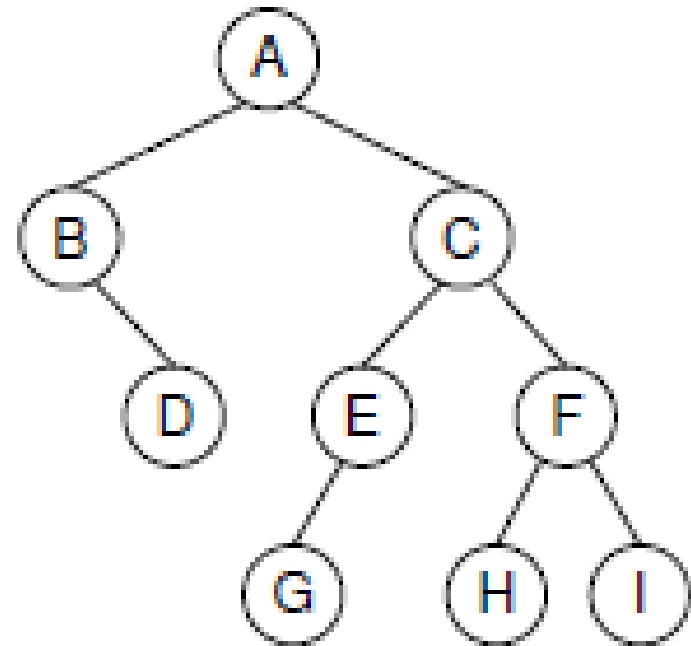
5. E

6. C

7. 

8. 

9. 



In-Order Traversal

1. B

2. D

3. A

4. G

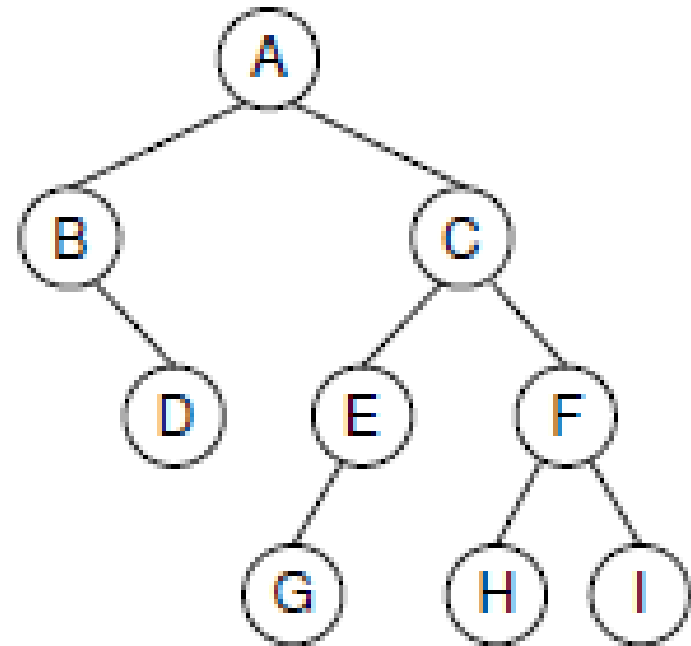
5. E

6. C

7. H

8.

9.



In-Order Traversal

1. B

2. D

3. A

4. G

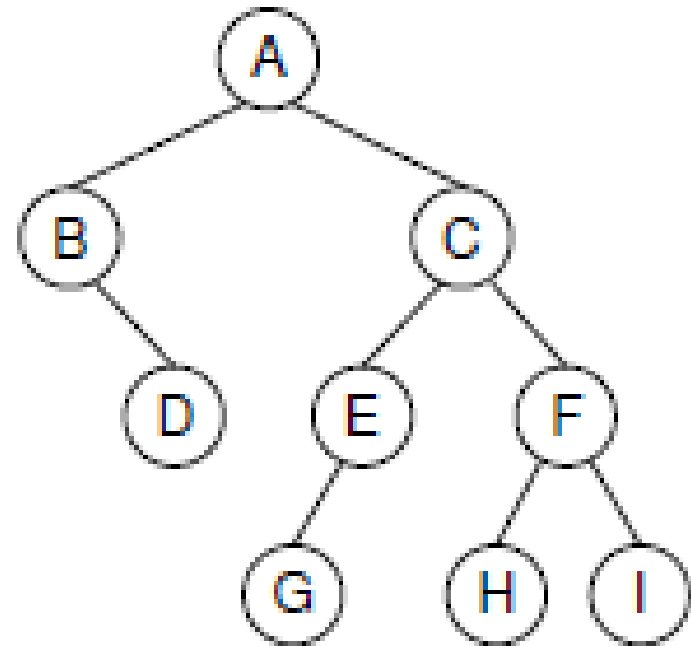
5. E

6. C

7. H

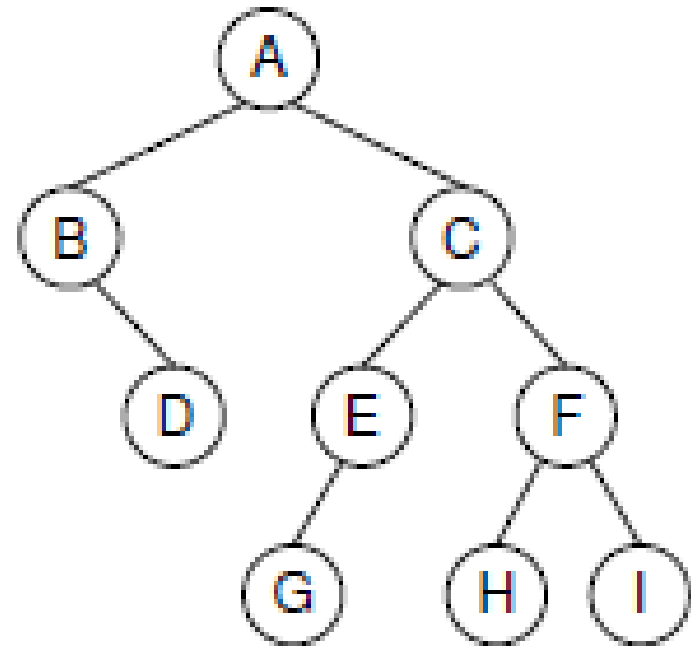
8. F

9. 



In-Order Traversal

1. B
2. D
3. A
4. G
5. E
6. C
7. H
8. F
9. I



Pre-Order Traversal Code V1

```
/** @param rt The root of the subtree */  
void preorder(BinNode rt)  
{  
    if (rt == null) return; // Empty subtree  
    visit(rt);  
    preorder(rt.left());  
    preorder(rt.right());  
}
```

Pre-Order Traversal Code V2

```
/** @param rt The root of the subtree */  
void preorder2(BinNode rt) // Not so good  
{  
    visit(rt);  
    if (rt.left() != null) preorder(rt.left());  
    if (rt.right() != null) preorder(rt.right());  
}
```

Preorder vs preorder2

- preorder2 makes only half as many recursive calls. (Why?)
- On the other hand, **preorder2 must access the left and right child pointers twice as often.**
- **The net result is little or no performance improvement.**

```
void preorder(BinNode rt)
{
    if (rt == null) return; // Empty subtree
    visit(rt);
    preorder(rt.left());
    preorder(rt.right());
}

void preorder2(BinNode rt) // Not so good
{
    visit(rt);
    if (rt.left() != null) preorder(rt.left());
    if (rt.right() != null) preorder(rt.right());
}
```

Preorder2 is inferior

- First, for more complex traversals it can become awkward to place the check for the **NULL pointer in the** calling code.
 - Even here we had to write two tests for **NULL**, rather than the one needed by **preorder**.
- The more important concern with **preorder2** is **that it** tends to be error prone.
 - While **preorder2** insures that no recursive calls will be made on empty subtrees, it will fail if the initial call passes in a **NULL pointer**.

Recursion Example: Counting the nodes

- We wish to count the number of nodes in a binary tree.
 - How?
- The key insight is that the total count for any (non-empty) subtree is one for the root plus the counts for the left and right subtrees.

```
int count(BinNode rt) {  
    if (rt == null) return 0;  
    return 1 + count(rt.left()) +  
            count(rt.right());  
}
```

Binary Tree Implementation: 'Key' Value

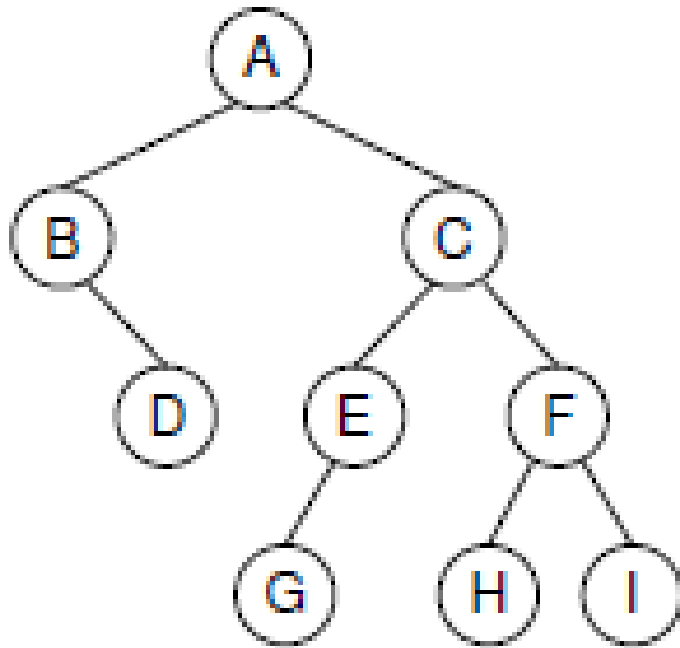
- We do not want to describe what we are looking for by detailing and matching the entire contents of the record.
- If we knew everything about the record already, we probably would not need to look for it.
- Instead, we typically define what record we want in terms of a key value.
- For example, if searching for payroll records, we might wish to search for the record that matches a particular ID number.
- The ID number is then the search 'key'.

Binary Tree Implementation

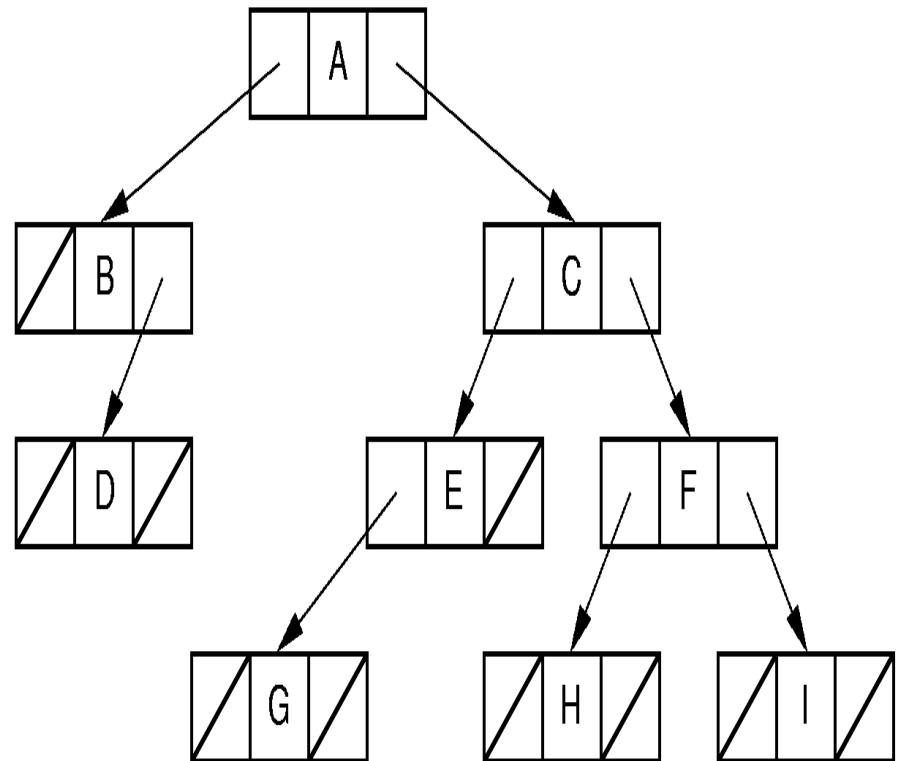
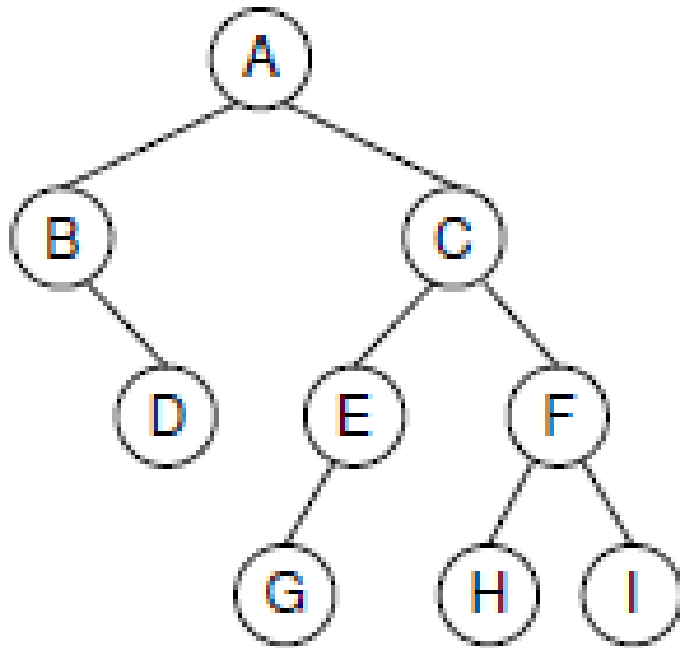


C++3elatestP161.pdf

Binary Tree Implementation



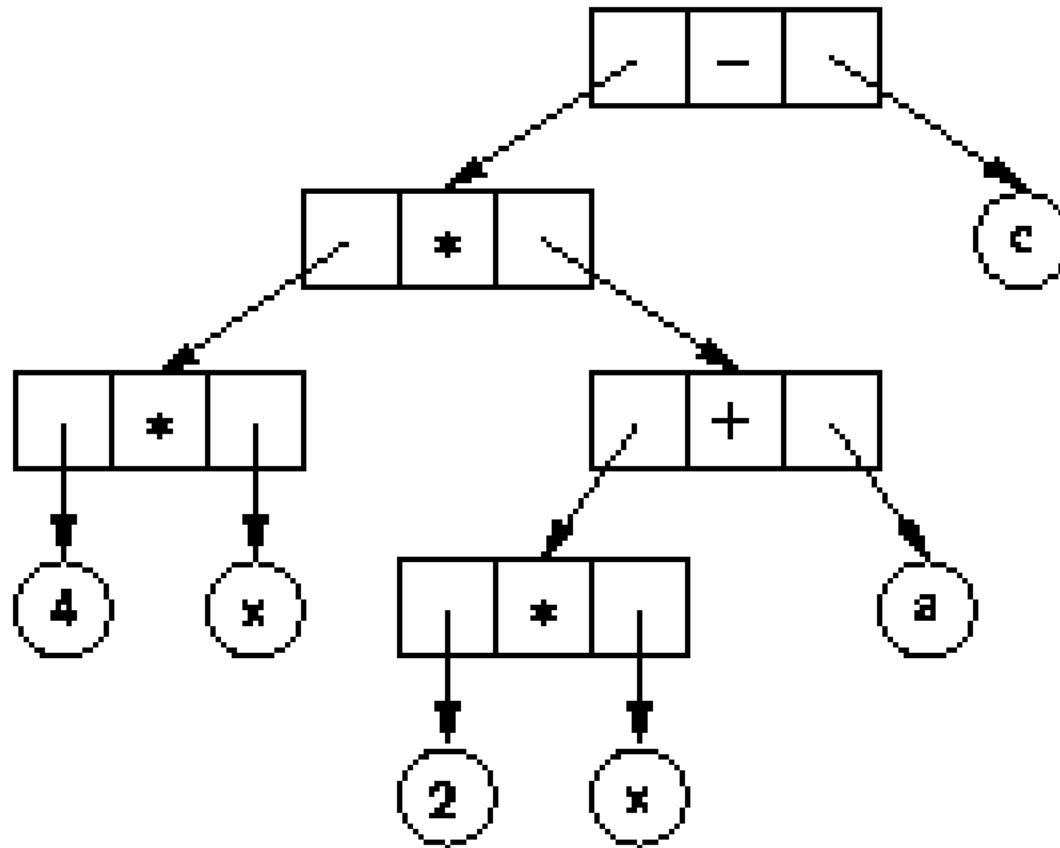
Binary Tree Implementation



Some important issues

- whether the same class definition will be used for leaves and internal nodes.
 - Using the same class for both will simplify the implementation,
 - but might be an inefficient use of space.
 - Some applications require data values only for the leaves.
 - Other applications require one type of value for the leaves and another for the internal nodes.
 - Also, it seems wasteful to store child pointers in the leaf nodes.

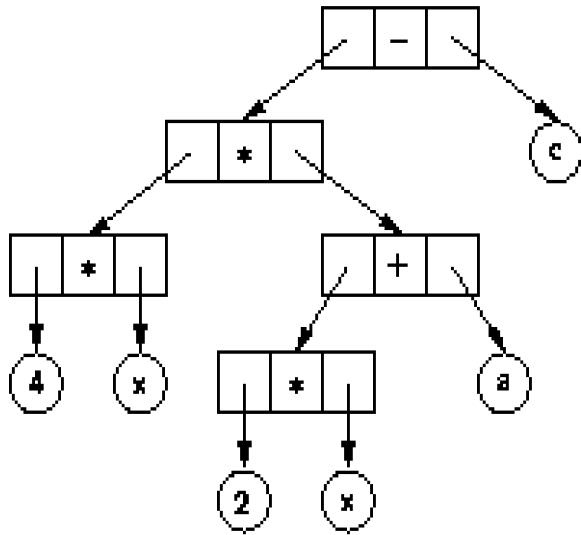
Binary Tree Implementation



$$4x(2x + a) + c$$

$$4 * x * (2 * x + a) + c$$

Binary Tree Implementation



$$4x(2x + a) + c$$

$$4 * x * (2 * x + a) + c$$

- Internal nodes store operators
 - could store a small code identifying the operator (a single byte for the operator's symbol)
- the leaves store operands
 - i.e., variable names or numbers, (considerably larger in order to handle the wider range of possible values)
 - No child pointers though

Different nodes implementation with inheritance (1)



C++3elatestP164.pdf

Different nodes implementation with inheritance (2)



C++3elatestP165.pdf

Why two Implementations?

[Design Patterns]

- Design patterns capture reusable pieces of design wisdom.
- Goals:
 - Quickly communicate design wisdom to new designers
 - Give a shared vocabulary to designers
- WE WILL NOT FOCUS ON THIS

Space Requirements

- Every node has two pointers to its children
- total space = $n(2P + D)$ for a tree of n nodes
 - P : space required by a pointer
 - D : amount of space required by a data value
- So, total overhead: $2Pn$
- Overhead fraction: $2P/(2P+D)$
- $P = D \Rightarrow 2/3^{\text{rd}}$ of its total space is overhead
- From the Full Binary Tree Theorem: Half of the pointers are **null**.
 - half of the pointers are “wasted” **NULL values that serve only to indicate tree** structure, but which do not provide access to new data.

Space Overhead

- A common implementation is not to store any actual data in a node
 - but rather a pointer to the data record.
- In this case, each node will typically store three pointers all of which are overhead:
 - overhead fraction of $3P/(3P + D)$
 - $P = D \Rightarrow 3/4^{\text{th}}$ of its total space is overhead

Space Overhead

- If only leaves store data values, then the fraction of total space devoted to overhead depends on whether the tree is full.
 - If the tree is not full, then conceivably there might only be one leaf node at the end of a series of internal nodes.
 - Thus, the overhead can be an arbitrarily high percentage
- The overhead fraction drops as the tree becomes closer to full, being lowest when the tree is truly full.
 - In this case, about one half of the nodes are internal.

Space Overhead

Eliminate pointers from the leaf nodes:

$$\frac{n/2(2P)}{n/2(2P) + Dn} = \frac{P}{P + D}$$

This is 1/2 if $P = D$.

Full Binary Tree

$n/2$ IN has $2P$
0 P in L
 $n/2$ IN has D
 $\sim n/2$ L has D

Space Overhead

If data only at leaves with
pointers eliminated

$(2P)/(2P + D) \Rightarrow 2/3$
overhead (Assuming $P=D$).

Full Binary Tree

$n/2$ IN has $2P$
 $\sim n/2$ L has D

*Note that some method is
needed to distinguish
leaves from internal nodes.
So some space is needed
here which we ignore in our
analysis*

Space Overhead

- a better implementation would have the internal nodes store two pointers and no data field while the leaf nodes store only a pointer to the data field.

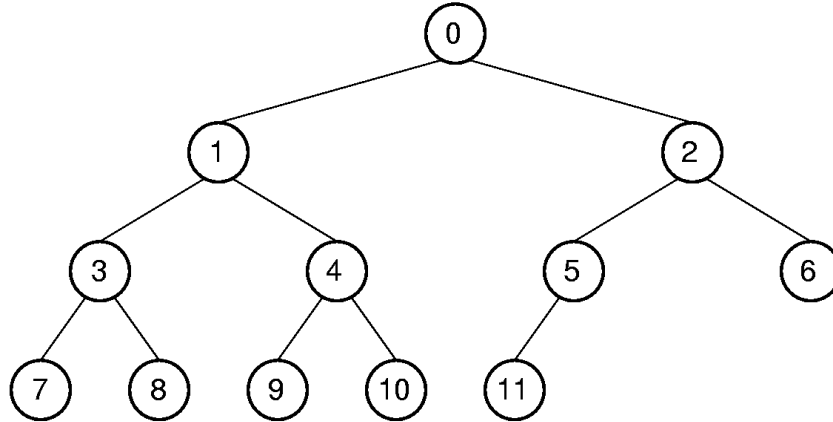
– $(3P)/(3P + D) \Rightarrow 3/4$ when $D = P$.

Full Binary Tree

$n/2$ IN has $2P$
 $\sim n/2$ L has $1P$
 $\sim n/2$ L has $1D$

Array Implementation

Complete Binary Tree

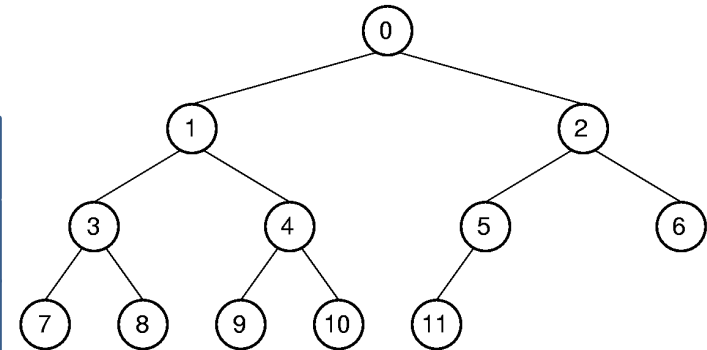


Position	0	1	2	3	4	5	6	7	8	9	10	11
Parent	--	0	0	1	1	2	2	3	3	4	4	5
Left Child	1	3	5	7	9	11	--	--	--	--	--	--
Right Child	2	4	6	8	10	--	--	--	--	--	--	--
Left Sibling	--	--	1	--	3	--	5	--	7	--	9	--
Right Sibling	--	2	--	4	--	6	--	8	--	10	--	--

Array Implementation

Position	0	1	2	3	4	5	6	7	8	9	10	11
Parent	--	0	0	1	1	2	2	3	3	4	4	5
Left Child	1	3	5	7	9	11	--	--	--	--	--	--
Right Child	2	4	6	8	10	--	--	--	--	--	--	--
Left Sibling	--	--	1	--	3	--	5	--	7	--	9	--
Right Sibling	--	2	--	4	--	6	--	8	--	10	--	--

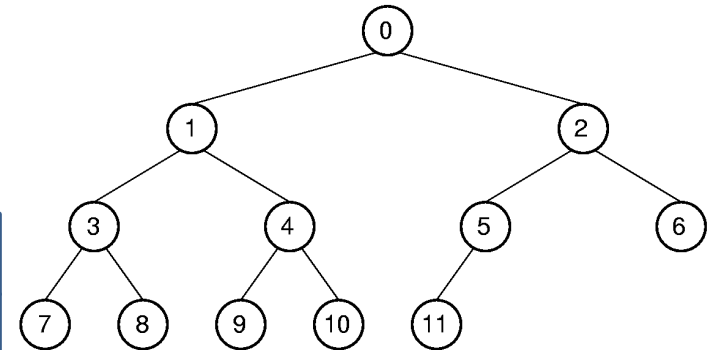
- $\text{Parent}(r) = \lfloor (r - 1) / 2 \rfloor$ if $r \neq 0$.



Array Implementation

Position	0	1	2	3	4	5	6	7	8	9	10	11
Parent	--	0	0	1	1	2	2	3	3	4	4	5
Left Child	1	3	5	7	9	11	--	--	--	--	--	--
Right Child	2	4	6	8	10	--	--	--	--	--	--	--
Left Sibling	--	--	1	--	3	--	5	--	7	--	9	--
Right Sibling	--	2	--	4	--	6	--	8	--	10	--	--

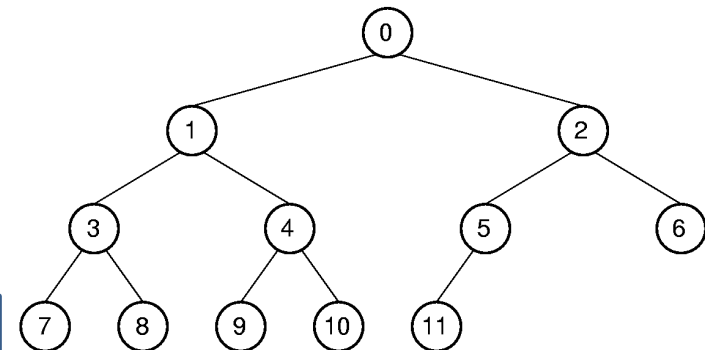
- $\text{Parent}(r) = \lfloor (r - 1) / 2 \rfloor$ if $r \neq 0$.
- $\text{Left child}(r) = 2r + 1$ if $2r + 1 < n$.



Array Implementation

Position	0	1	2	3	4	5	6	7	8	9	10	11
Parent	--	0	0	1	1	2	2	3	3	4	4	5
Left Child	1	3	5	7	9	11	--	--	--	--	--	--
Right Child	2	4	6	8	10	--	--	--	--	--	--	--
Left Sibling	--	--	1	--	3	--	5	--	7	--	9	--
Right Sibling	--	2	--	4	--	6	--	8	--	10	--	--

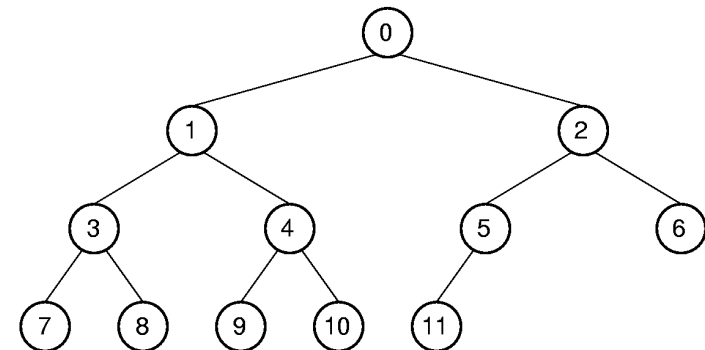
- $\text{Parent}(r) = \lfloor (r - 1) / 2 \rfloor$ if $r \neq 0$.
- $\text{Left child}(r) = 2r + 1$ if $2r + 1 < n$.
- $\text{Right child}(r) = 2r + 2$ if $2r + 2 < n$.



Array Implementation

Position	0	1	2	3	4	5	6	7	8	9	10	11
Parent	--	0	0	1	1	2	2	3	3	4	4	5
Left Child	1	3	5	7	9	11	--	--	--	--	--	--
Right Child	2	4	6	8	10	--	--	--	--	--	--	--
Left Sibling	--	--	1	--	3	--	5	--	7	--	9	--
Right Sibling	--	2	--	4	--	6	--	8	--	10	--	--

- $\text{Parent}(r) = \lfloor (r - 1) / 2 \rfloor$ if $r \neq 0$.
- $\text{Left child}(r) = 2r + 1$ if $2r + 1 < n$.
- $\text{Right child}(r) = 2r + 2$ if $2r + 2 < n$.
- $\text{Left sibling}(r) = r - 1$ if r is even.



Array Implementation

Position	0	1	2	3	4	5	6	7	8	9	10	11
Parent	--	0	0	1	1	2	2	3	3	4	4	5
Left Child	1	3	5	7	9	11	--	--	--	--	--	--
Right Child	2	4	6	8	10	--	--	--	--	--	--	--
Left Sibling	--	--	1	--	3	--	5	--	7	--	9	--
Right Sibling	--	2	--	4	--	6	--	8	--	10	--	--

- $\text{Parent}(r) = \lfloor (r - 1) / 2 \rfloor$ if $r \neq 0$.
- $\text{Left child}(r) = 2r + 1$ if $2r + 1 < n$.
- $\text{Right child}(r) = 2r + 2$ if $2r + 2 < n$.
- $\text{Left sibling}(r) = r - 1$ if r is even.
- $\text{Right sibling}(r) = r + 1$ if r is odd and $r + 1 < n$.

